

Computer Organization and Architecture

Module 5

Computer Arithmetic

Department of Computer Science and Engineering

IIT Kharagpur

Computer Arithmetic

Introduction

- Computers are built using tiny electronic switches.
 - Typically made up of MOS transistors.
 - The state of the switches are typically expressed in binary (ON/OFF).
- To design arithmetic circuits for use in computers, we need to work with *binary numbers*.
 - How to carry out various arithmetic operations in binary?
 - How to implement them efficiently in hardware?

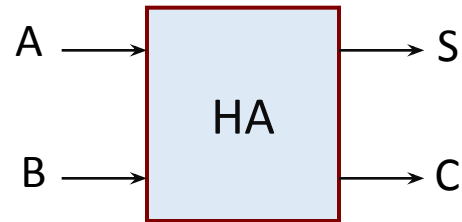
Addition / Subtraction

Addition of Two Binary Digits (Bits)

- When two bits A and B are added, a sum (S) and carry (C) are generated as per the following truth table:

Inputs		Outputs	
A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

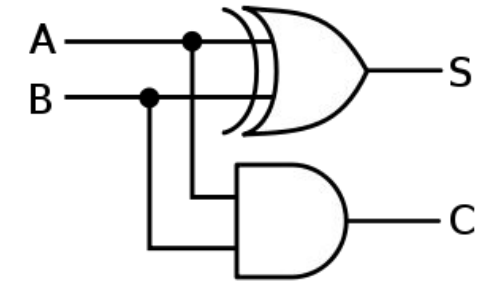
$0 + 0 = 00$
 $0 + 1 = 01$
 $1 + 0 = 01$
 $1 + 1 = 10$



HALF ADDER

$$S = A'.B + A.B' = A \oplus B$$

$$C = A.B$$



Addition of Multi-bit Binary Numbers

$$\begin{array}{r} 0010110 \leftarrow \text{Carry} \\ 0101011 \leftarrow \text{Number A} \\ + 0001001 \leftarrow \text{Number B} \\ \hline 0110100 \leftarrow \text{Sum S} \end{array}$$

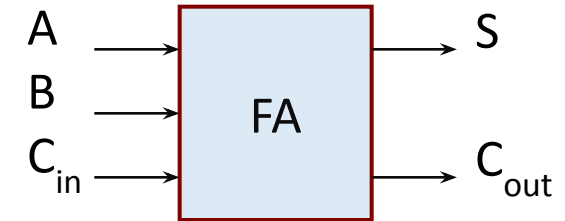
$$\begin{array}{r} 1111110 \leftarrow \text{Carry} \\ 0111111 \leftarrow \text{Number A} \\ + 0000001 \leftarrow \text{Number B} \\ \hline 1000000 \leftarrow \text{Sum S} \end{array}$$

- At every bit position (stage), we require to add 3 bits:
 - 1 bit for number A
 - 1 bit for number B
 - 1 carry bit coming from the previous stage

WE NEED A FULL ADDER

Full Adder

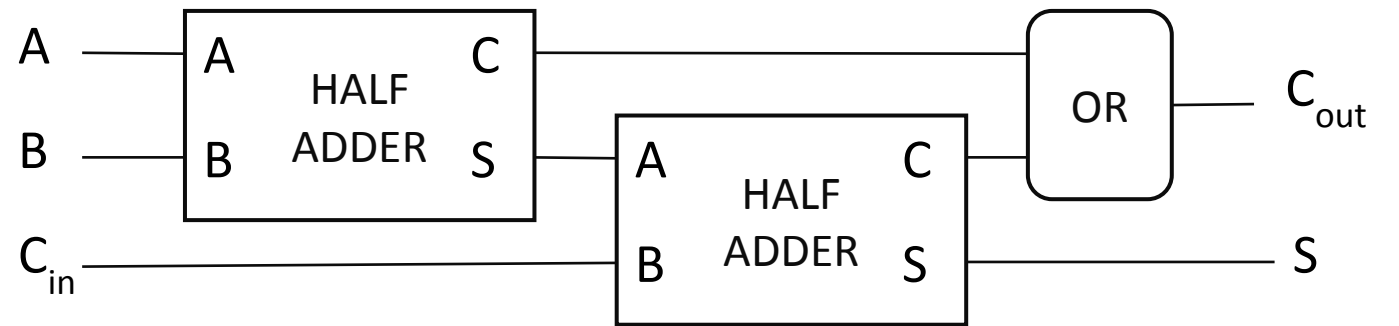
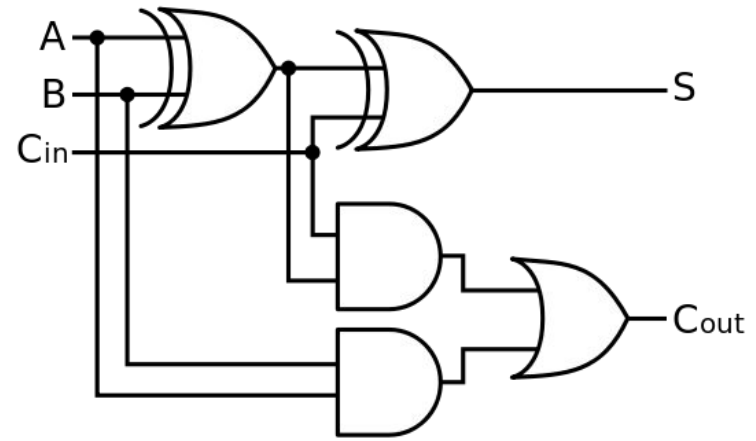
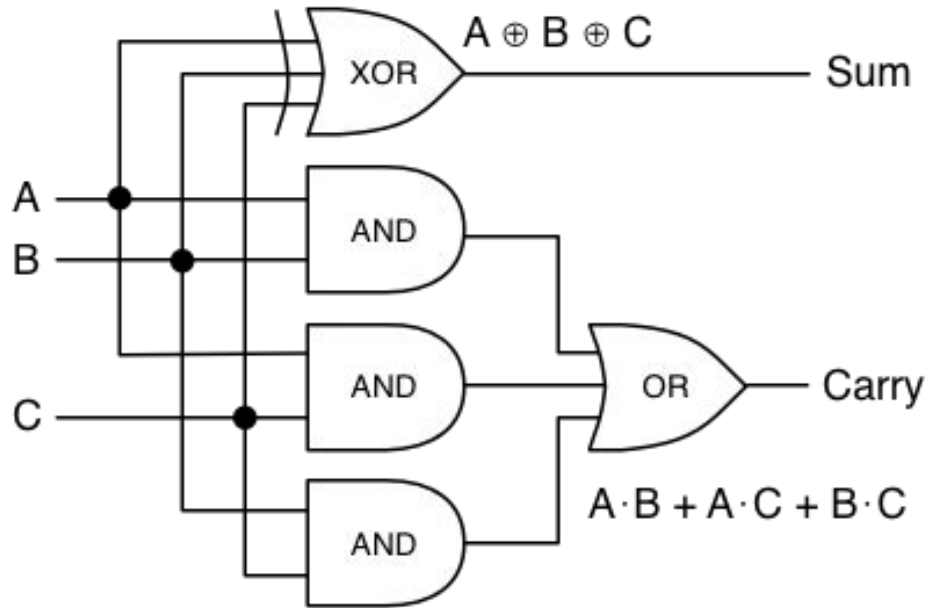
Inputs			Outputs	
A	B	C _{in}	S	C _{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



$$\begin{aligned}
 S &= A'.B'.C_{in} + A'.B.C_{in}' + A.B'.C_{in}' + A.B.C_{in} \\
 &= A \oplus B \oplus C_{in}
 \end{aligned}$$

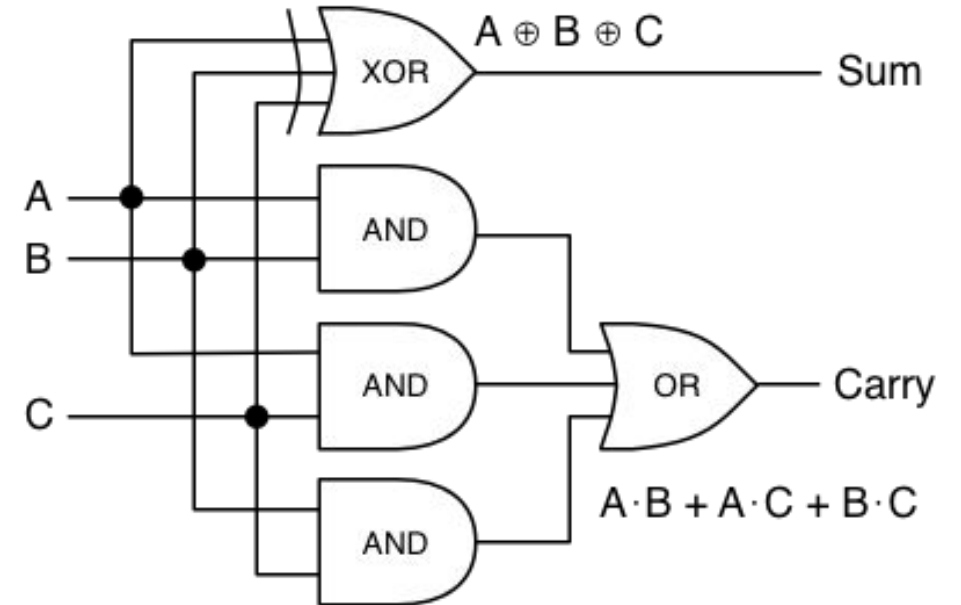
$$\begin{aligned}
 C_{out} &= B.C_{in} + A.C_{in} + A.B + A.B.C_{in} \\
 &= A.B + B.C_{in} + A.C_{in}
 \end{aligned}$$

Various Implementations of Full Adder



• Delay of a full adder:

- Assume that the delay of all basic gates (AND, OR, NAND, NOR, NOT) is δ
- Delay for Carry = 2δ
- Delay for Sum = 3δ
(AND-OR delay plus one inverter delay)



Parallel Adder Design

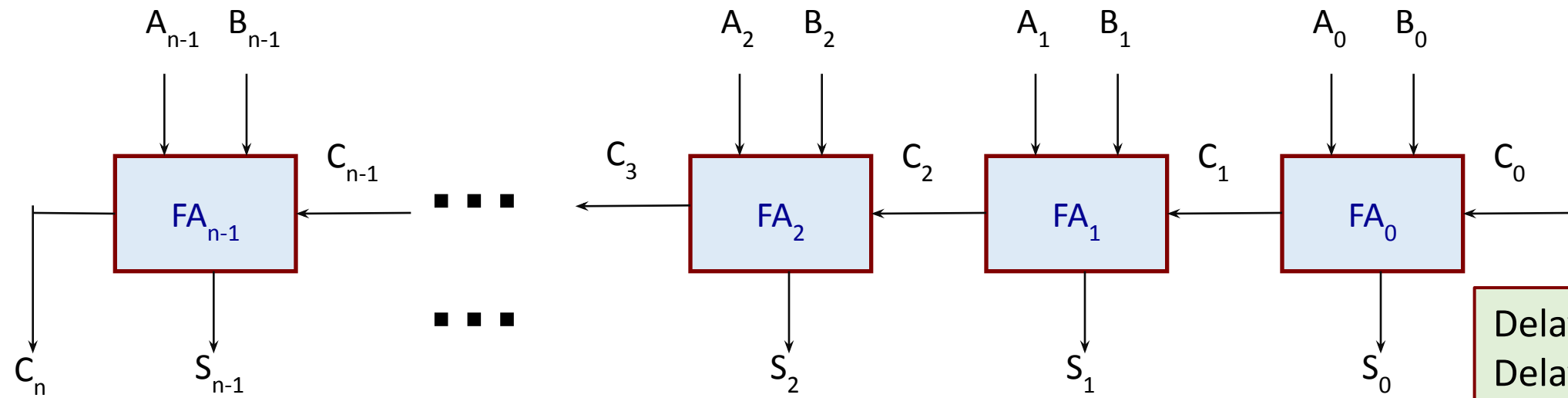
- We shall look at the various designs of n -bit parallel adder.
 - a) Ripple carry adder
 - b) Carry look-ahead adder
 - c) Carry save adder
 - d) Carry select adder

Ripple Carry Adder

- Cascade n full adders to create a n -bit parallel adder.
- Carry output from stage- i propagates as the carry input to stage- $(i+1)$.
- In the worst-case, carry ripples through all the stages.

The diagram illustrates a ripple carry adder with 8 bits. It shows the addition of two numbers, A and B, with an initial carry-in of 1. The carry ripples from right to left through all stages, resulting in a final carry-out of 1. Blue arrows indicate the direction of carry propagation from right to left.

1	1	1	1	1	1	1	0	Carry
0	1	1	1	1	1	1	1	Number A
+	0	0	0	0	0	0	1	Number B
1	0	0	0	0	0	0	0	Sum S



Delay for $C_1 = 2\delta$
 Delay for $C_2 = 4\delta$
 Delay for $C_{n-1} = 2(n-1)\delta$
 Delay for $C_n = 2n\delta$

Two numbers: $A_{n-1} \dots A_2 A_1 A_0$ and $B_{n-1} \dots B_2 B_1 B_0$
 Input carry: C_0
 Sum: $S_{n-1} \dots S_2 S_1 S_0$
 Output carry: C_n

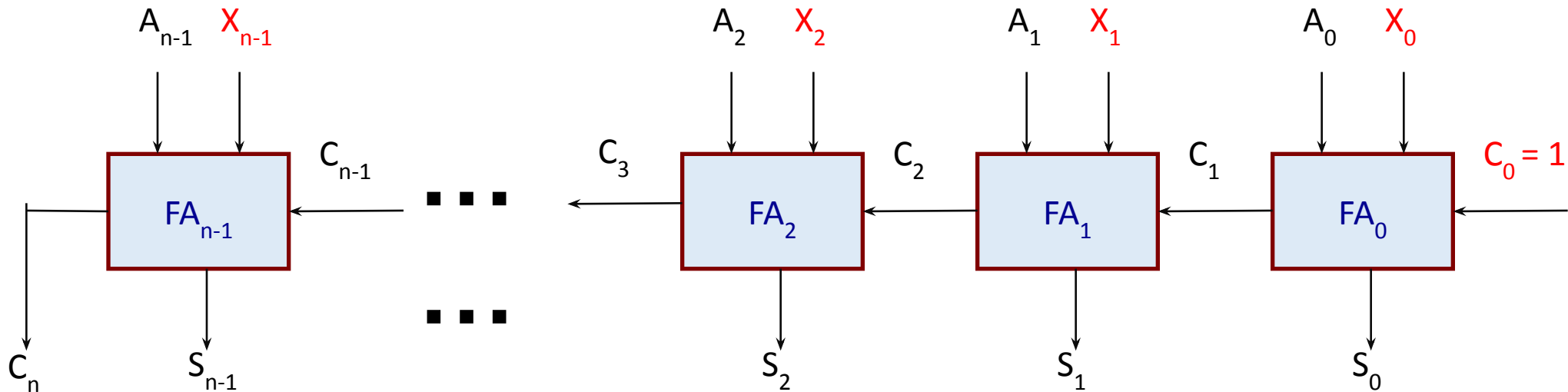
Delay for $S_0 = 3\delta$
 Delay for $S_1 = 2\delta + 3\delta = 5\delta$
 Delay for $S_2 = 4\delta + 3\delta = 7\delta$
 Delay for $S_{n-1} = 2(n-1)\delta + 3\delta = (2n+1)\delta$

Delay is proportional to n

How to Design a Parallel Subtractor?

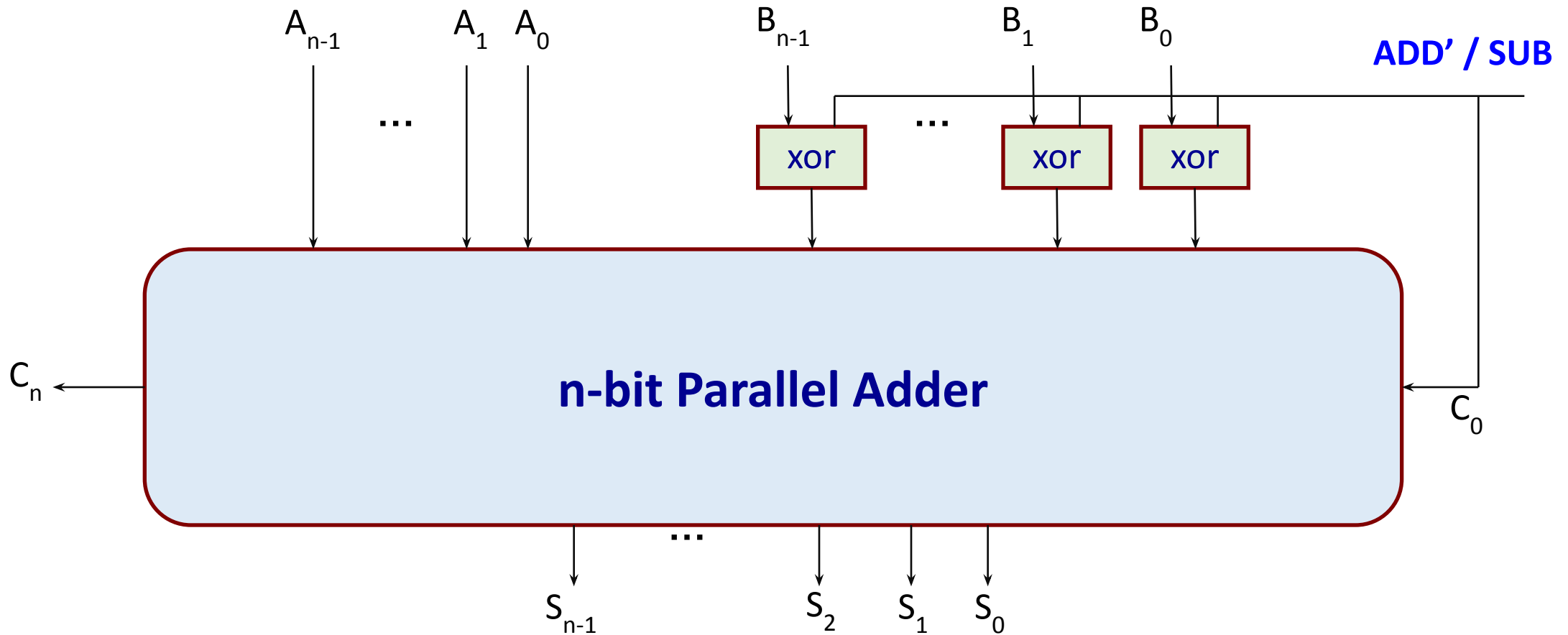
- **Observation:**

- Computing $A - B$ is the same as adding the 2's complement of B to A .
- 2's complement is equal to 1's complement plus 1.
- Let $X_i = B_i'$.



A Parallel Adder/Subtractor

xor gate is a
controlled inverter



Carry Look-ahead Adder

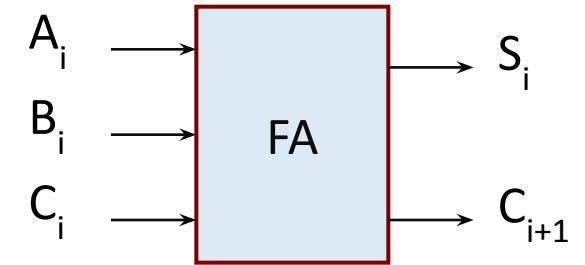
- The propagation delay of an n -bit ripple carry order has been seen to be proportional to n .
 - Due to the rippling effect of carry sequentially from one stage to the next.
- One possible way to speedup the addition.
 - Generate the carry signals for the various stages in parallel.
 - Time complexity reduces from $O(n)$ to $O(1)$.
 - However, hardware complexity increases rapidly with n .

- Consider the i -th stage in the addition process.
- We define the *carry generate* and *carry propagate* functions as:

$$G_i = A_i \cdot B_i$$

$$P_i = A_i \oplus B_i$$

- $G_i = 1$ represents the condition when a carry is generated in stage- i independent of the other stages.
- $P_i = 1$ represents the condition when an input carry C_i will be propagated to the output carry C_{i+1} .



$$C_{i+1} = G_i + P_i \cdot C_i$$

Unrolling the Recurrence

$$\begin{aligned}C_{i+1} &= G_i + P_i C_i = G_i + P_i (G_{i-1} + P_{i-1} C_{i-1}) = G_i + P_i G_{i-1} + P_i P_{i-1} C_{i-1} \\&= G_i + P_i G_{i-1} + P_i P_{i-1} (G_{i-2} + P_{i-2} C_{i-2}) \\&= G_i + P_i G_{i-1} + P_i P_{i-1} G_{i-2} + P_i P_{i-1} P_{i-2} C_{i-2} = \dots\end{aligned}$$

$$C_{i+1} = G_i + \sum_{k=0}^{i-1} G_k \prod_{j=k+1}^i P_j + C_0 \prod_{j=0}^i P_j$$

Design of 4-bit CLA Adder

$$C_4 = G_3 + G_2P_3 + G_1P_2P_3 + G_0P_1P_2P_3 + C_0P_0P_1P_2P_3$$

$$C_3 = G_2 + G_1P_2 + G_0P_1P_2 + C_0P_0P_1P_2$$

$$C_2 = G_1 + G_0P_1 + C_0P_0P_1$$

$$C_1 = G_0 + C_0P_0$$

$$S_0 = A_0 \oplus B_0 \oplus C_0 = P_0 \oplus C_0$$

$$S_1 = P_1 \oplus C_1$$

$$S_2 = P_2 \oplus C_2$$

$$S_3 = P_3 \oplus C_3$$

4 AND2 gates

3 AND3 gates

2 AND4 gates

1 AND5 gate

1 OR2, 1 OR3, 1 OR4 and

1 OR5 gate

4 XOR2 gates

Design of 4-bit CLA Adder

$$C_4 = G_3 + G_2 P_3 + G_1 P_2 P_3 + G_0 P_1 P_2 P_3 + C_0 P_0 P_1 P_2 P_3$$

$$C_3 = G_2 + G_1 P_2 + G_0 P_1 P_2 + C_0 P_0 P_1 P_2$$

$$C_2 = G_1 + G_0 P_1 + C_0 P_0 P_1$$

$$C_1 = G_0 + C_0 P_0$$

$$S_0 = A_0 \oplus B_0 \oplus C_0 = P_0 \oplus C_0$$

$$S_1 = P_1 \oplus C_1$$

$$S_2 = P_2 \oplus C_2$$

$$S_3 = P_3 \oplus C_3$$

4 AND2 gates

3 AND3 gates

2 AND4 gates

1 AND5 gate

1 OR2, 1 OR3, 1 OR4 and

1 OR5 gate

4 XOR2 gates

Design of 4-bit CLA Adder

$$C_4 = G_3 + \mathbf{C}_3 P_3$$

$$C_3 = G_2 + G_1 P_2 + G_0 P_1 P_2 + C_0 P_0 P_1 P_2$$

$$C_2 = G_1 + G_0 P_1 + C_0 P_0 P_1$$

$$C_1 = G_0 + C_0 P_0$$

$$S_0 = A_0 \oplus B_0 \oplus C_0 = P_0 \oplus C_0$$

$$S_1 = P_1 \oplus C_1$$

$$S_2 = P_2 \oplus C_2$$

$$S_3 = P_3 \oplus C_3$$

4 AND2 gates

2 AND3 gates

1 AND4 gates

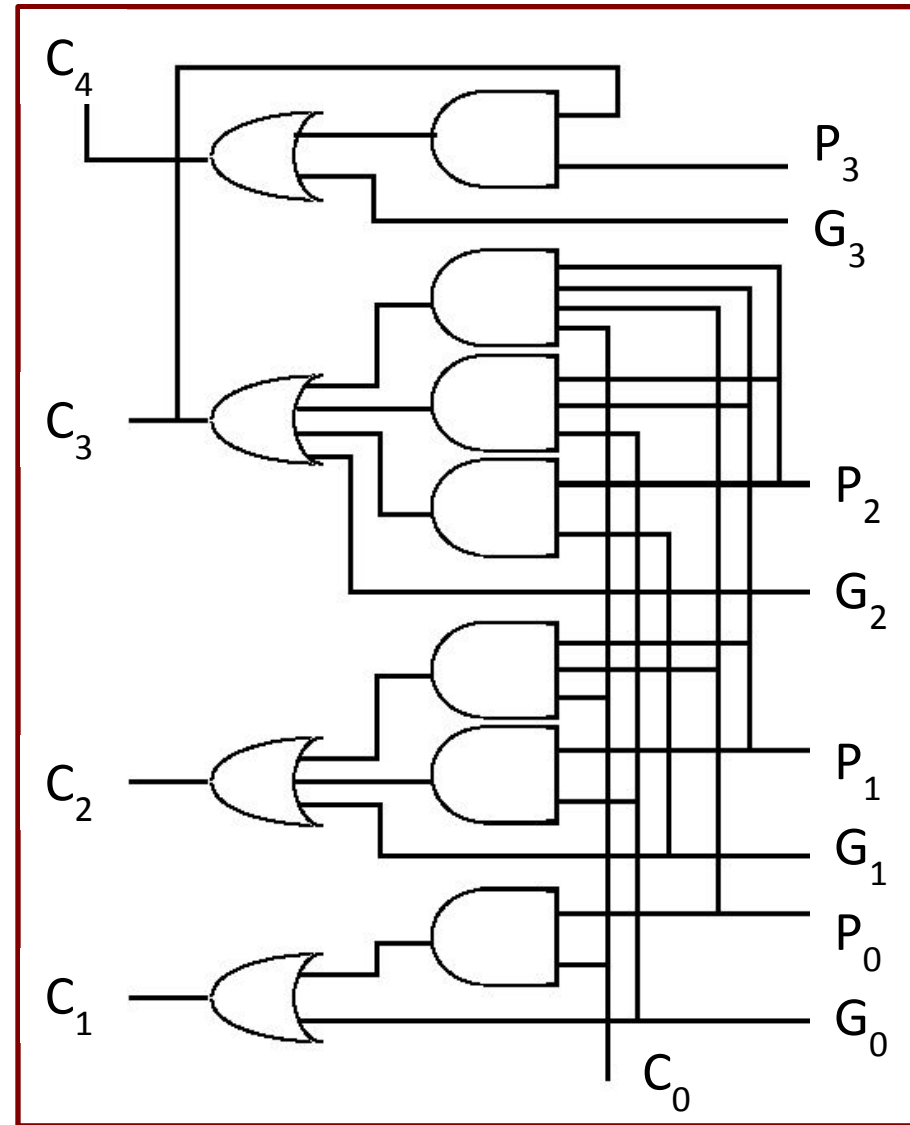
~~1 AND5 gate~~

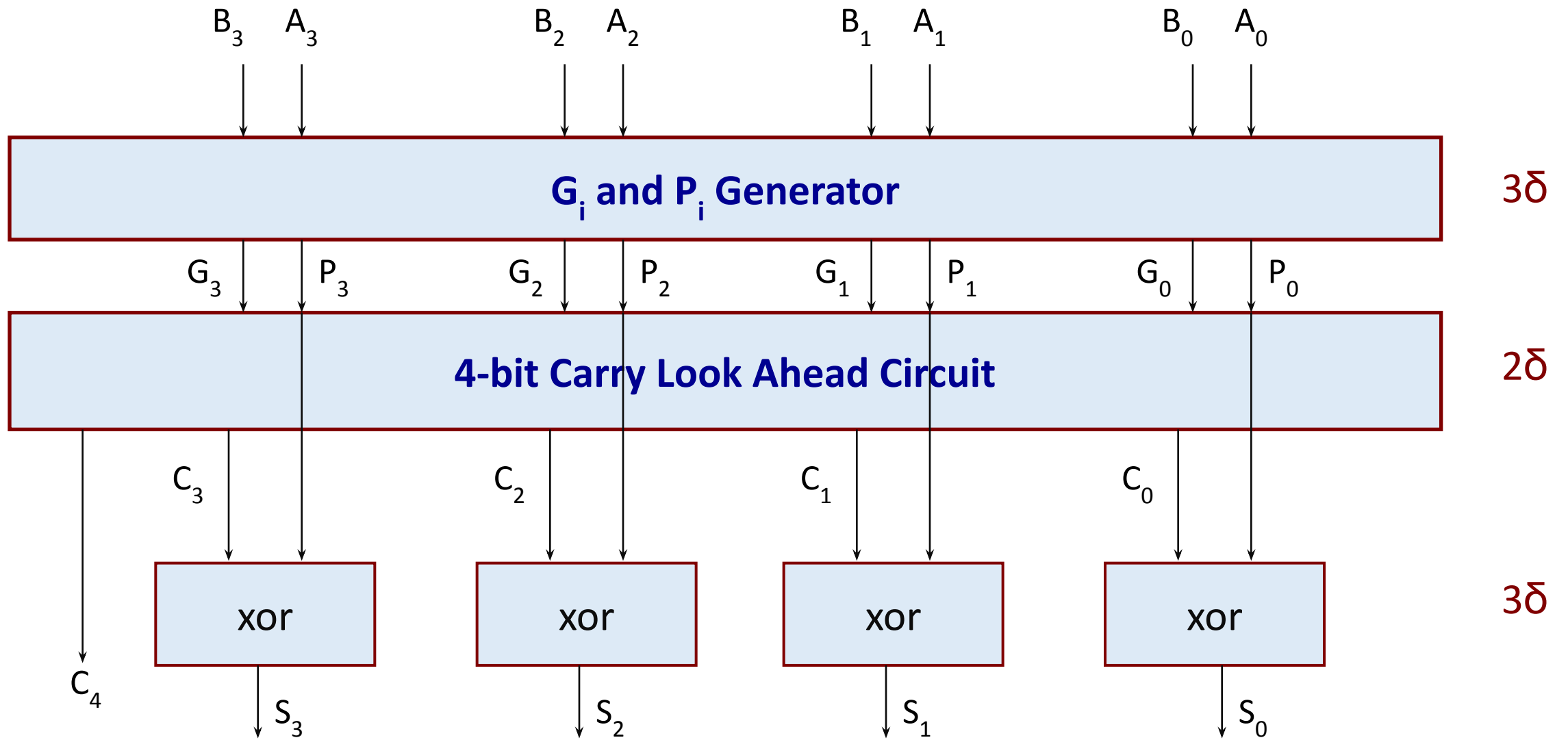
1 OR2, 1 OR3, 1 OR4 and

1 **OR2** gate

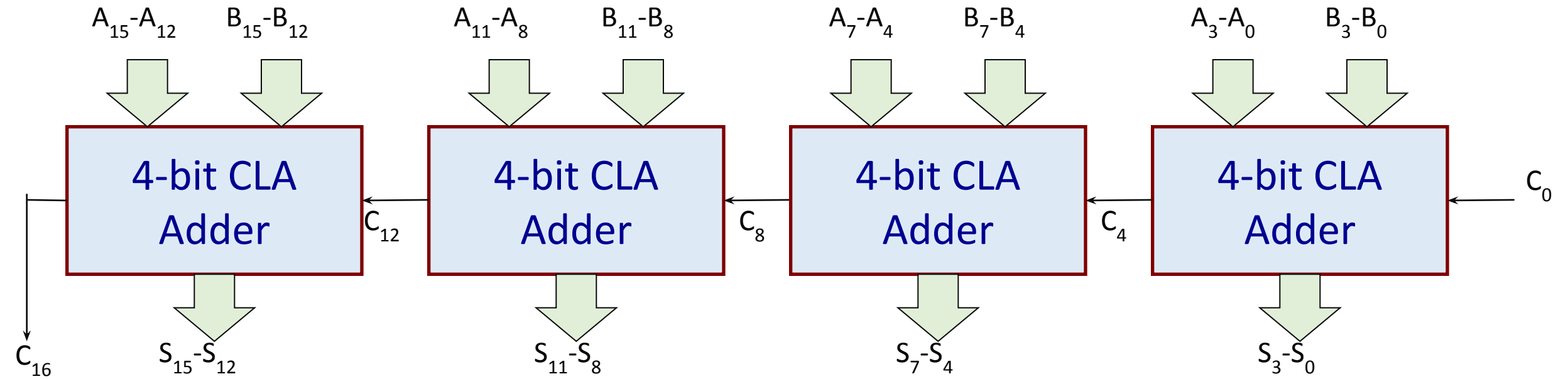
4 XOR2 gates

The 4-bit CLA Circuit





16-bit Adder Using 4-bit CLA Modules



Problem: Carry propagation between modules still slows down the adder

- **Solution:**

- Use a second level of carry look-ahead mechanism to generate the input carries to the CLA blocks in parallel.
- The second level of CLA generates C4, C8, C12 and C16 in parallel with two gate delays (2δ).
- For larger values of n , more CLA levels can be added.

- Delay calculation of a 16-bit adder:

- a) For original single-level CLA: 14δ
- b) For modified two-level CLA: 10δ

Delay of a k-bit Adder

n	T_{CLA}	T_{RCA}
4	8δ	9δ
16	10δ	33δ
32	12δ	65δ
64	12δ	129δ
128	14δ	257δ
256	14δ	513δ

$$T_{\text{CLA}} = (6 + 2\lceil \log_4 n \rceil) \delta$$

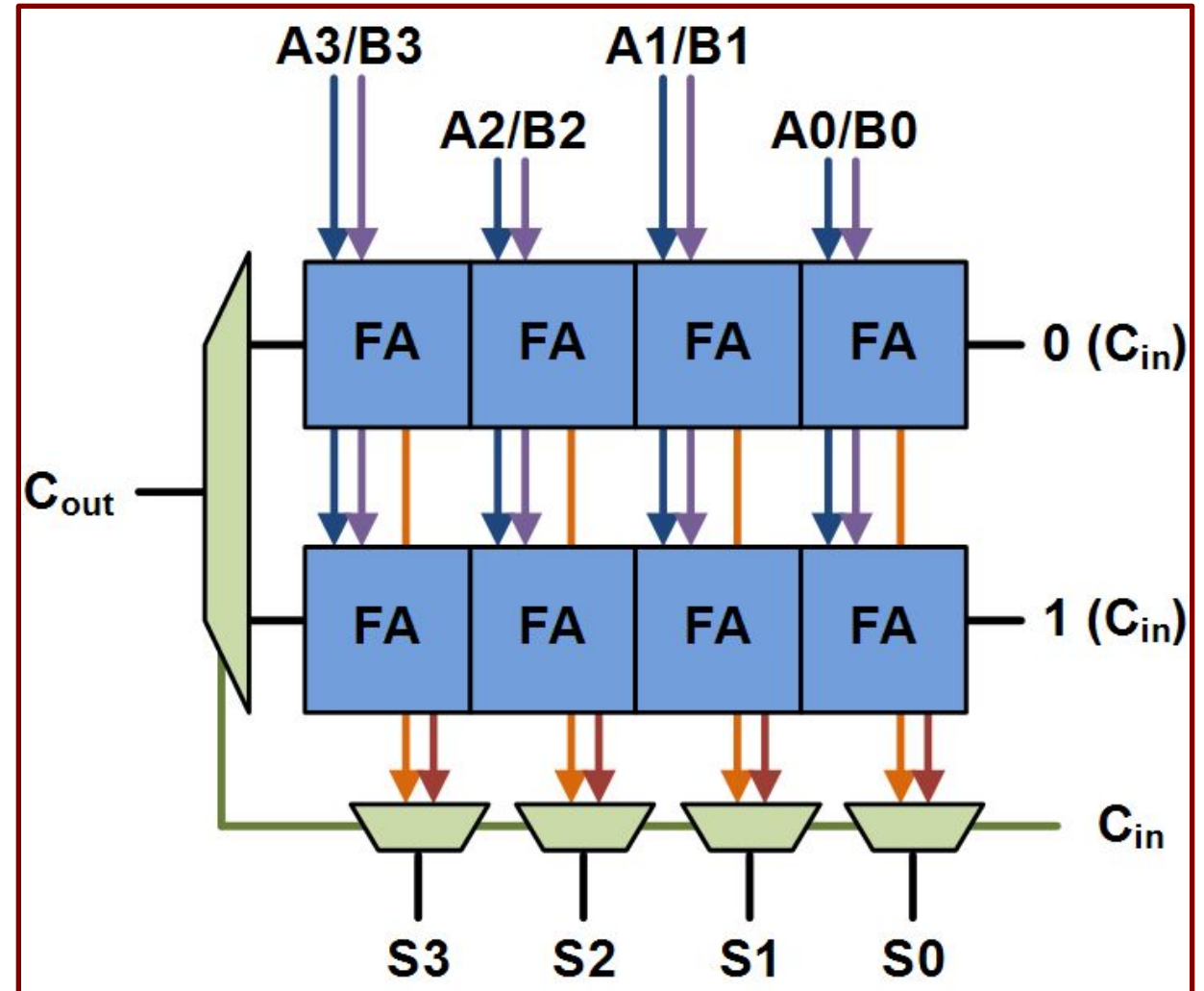
$$T_{\text{RCA}} = (2n + 1) \delta$$

Carry Select Adder

- Basically consists of two parallel adders (say, ripple-carry adder) and a multiplexer.
- For two given numbers A and B , we carry out addition twice:
 - With carry-in as 0
 - With carry-in as 1
- Once the correct carry-in is known, the correct sum is selected by a multiplexer.

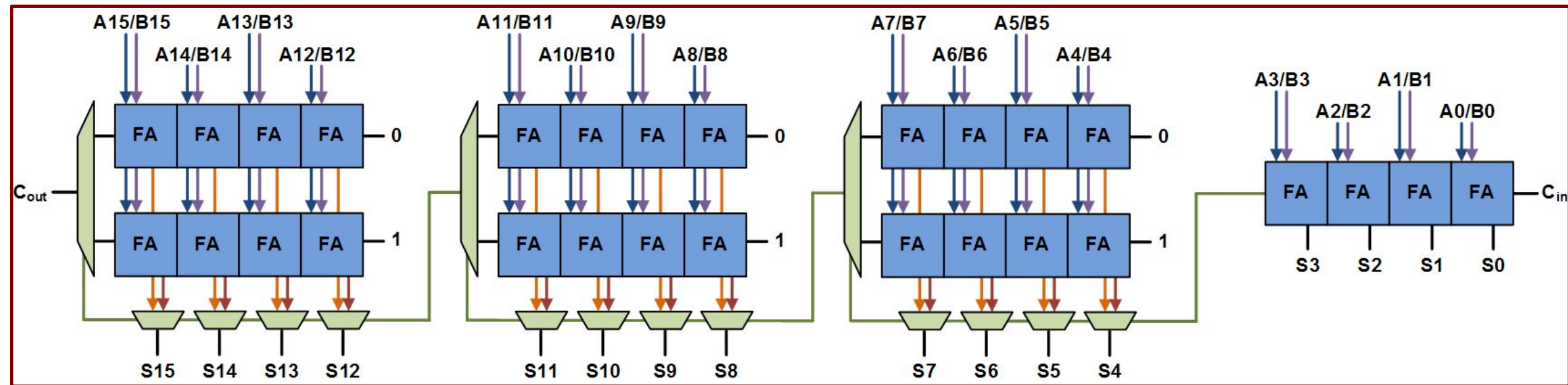
Basic building block of a carry-select adder, with block size of 4.

- For a multi-bit adder, the number of bits in each carry select block can be either uniform or variable.



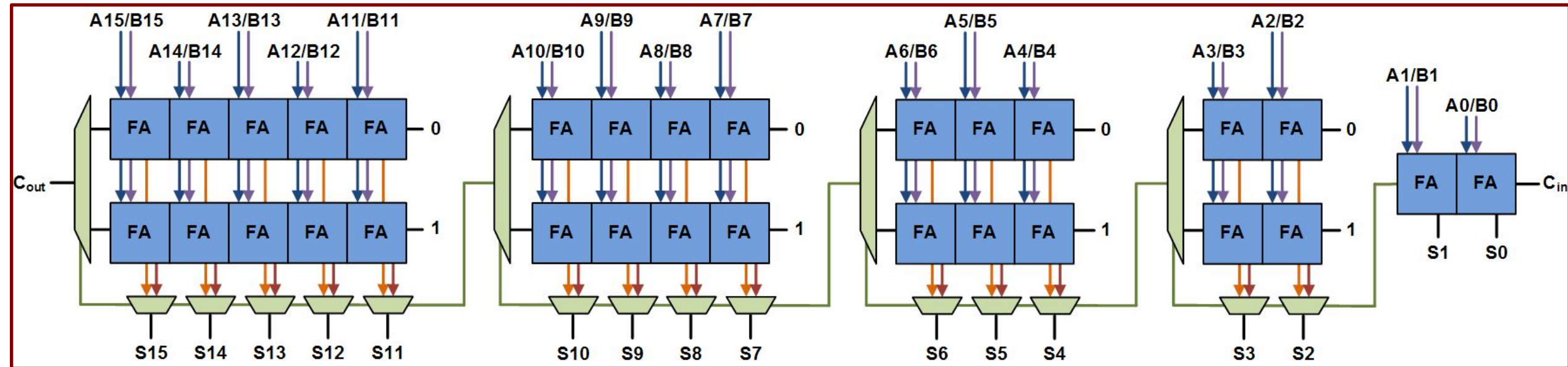
Uniform sized adder

- A 16-bit carry select adder with a uniform block size of 4 is shown.
- The least significant block needs a single adder (since the carry-in is known).
- Total delay is 4 full adder delays, plus 3 MUX delays.



Variable-sized adder

- A 16-bit carry select adder with variable block sizes of 2-2-3-4-5 is shown.
- Total delay is 2 full adder delays, plus 4 MUX delays.



Multiplication

Multiplication of Unsigned Numbers

- Multiplication requires substantially more hardware than addition.
- Multiplication of two n -bit number generates a $2n$ -bit product.
- We can use shift-and-add method.
 - Repeated additions of shifted versions of the multiplicand.

$\begin{array}{r} 1\ 0\ 1\ 0 \\ 1\ 1\ 0\ 1 \\ \hline 1\ 0\ 1\ 0 \\ 0\ 0\ 0\ 0 \\ 1\ 0\ 1\ 0 \\ 1\ 0\ 1\ 0 \\ \hline 1\ 0\ 0\ 0\ 0\ 0\ 1\ 0 \end{array}$	Multiplicand M (10) Multiplier Q (13) Product P (130)
--	--

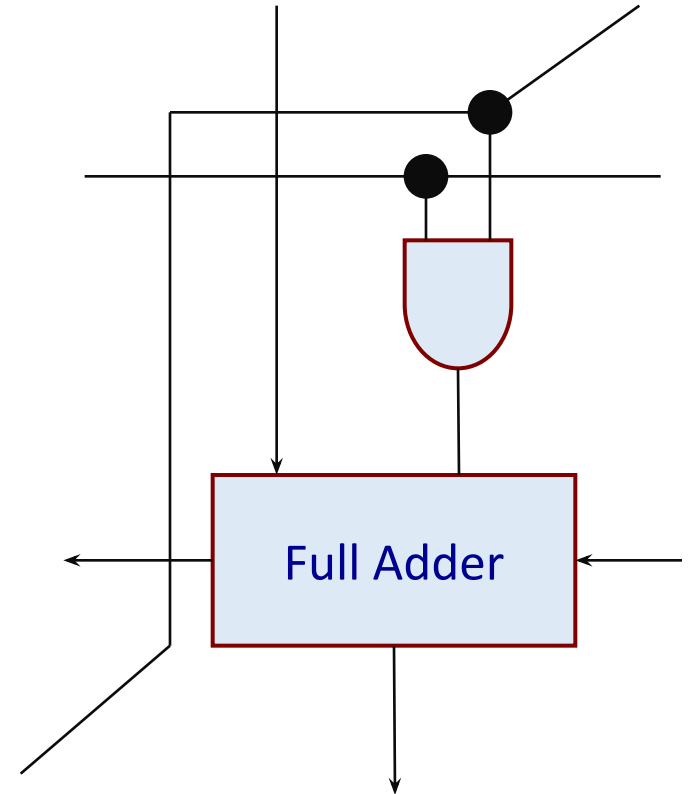
A General Case

$$\begin{array}{cccc}
 & & \mathbf{A}_3 & \mathbf{A}_2 & \mathbf{A}_1 & \mathbf{A}_0 \\
 & & \mathbf{B}_3 & \mathbf{B}_2 & \mathbf{B}_1 & \mathbf{B}_0 \\
 \hline
 & & \mathbf{A}_3\mathbf{B}_0 & \mathbf{A}_2\mathbf{B}_0 & \mathbf{A}_1\mathbf{B}_0 & \mathbf{A}_0\mathbf{B}_0 \\
 & \mathbf{A}_3\mathbf{B}_1 & \mathbf{A}_2\mathbf{B}_1 & \mathbf{A}_1\mathbf{B}_1 & \mathbf{A}_0\mathbf{B}_1 & \\
 & \mathbf{A}_3\mathbf{B}_2 & \mathbf{A}_2\mathbf{B}_2 & \mathbf{A}_1\mathbf{B}_2 & \mathbf{A}_0\mathbf{B}_2 & \\
 \mathbf{A}_3\mathbf{B}_3 & \mathbf{A}_2\mathbf{B}_3 & \mathbf{A}_1\mathbf{B}_3 & \mathbf{A}_0\mathbf{B}_3 & &
 \end{array}$$

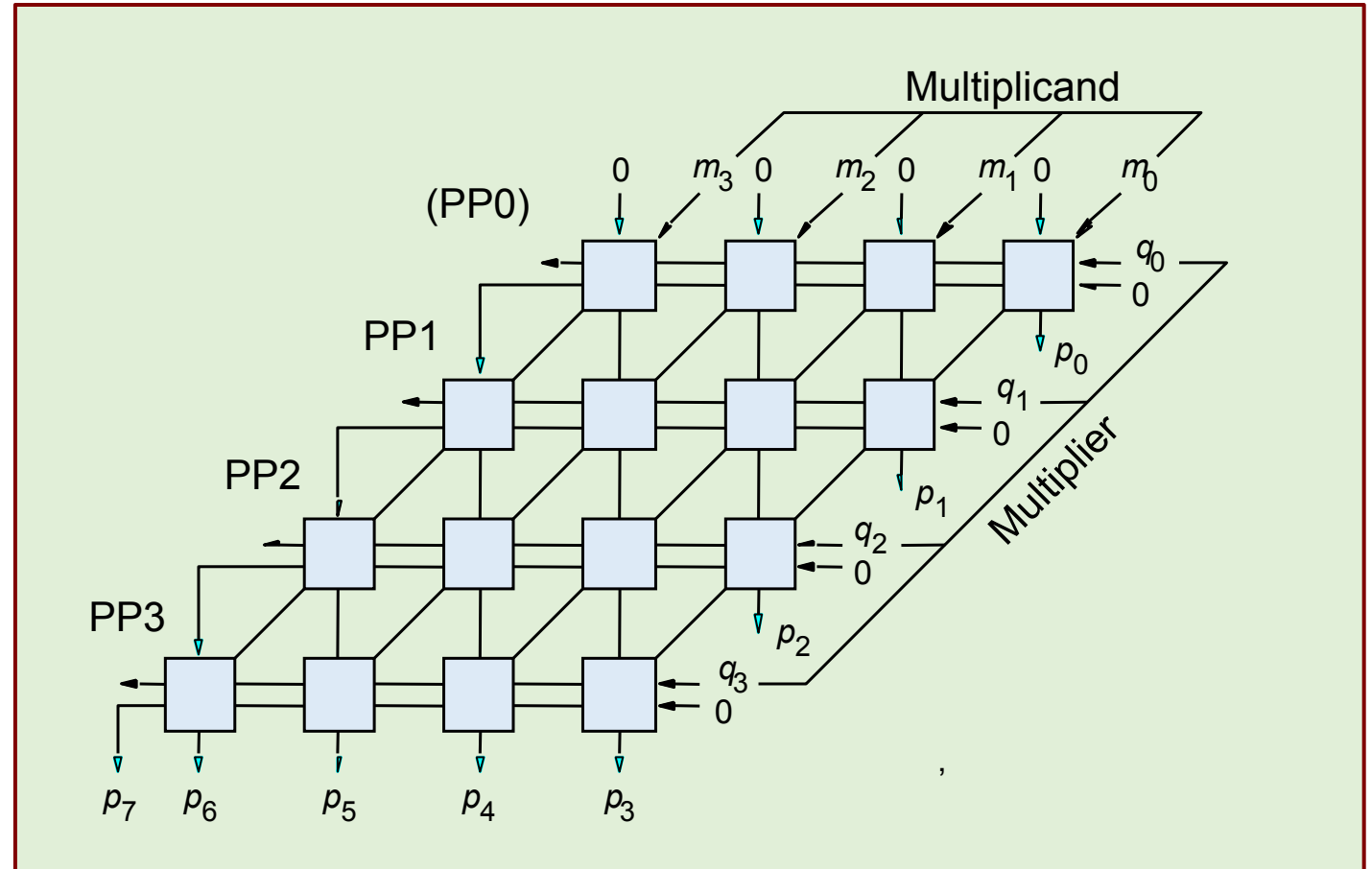
- Each $A_i B_j$ is called a *partial product*.
- Generating the partial products is easy.
 - Requires just an AND gate for each partial product.
- Adding all the n -bit partial products in hardware is more difficult.

Design of a Combinational Array Multiplier

- We can directly map the multiplication process as discussed to hardware.
 - We use an array of cells to generate the partial products.
 - Instead of adding the partial products at the end, we add the partial products at every stage of the multiplication.
- The required multiplication cell is as shown.
 - Combines capabilities of partial product generation and also addition of partial products.



- Extremely inefficient, and requires very large amount of hardware.
- Requires n^2 multiplication cells for an $n \times n$ multiplier.
- Advantage is that it is very fast.

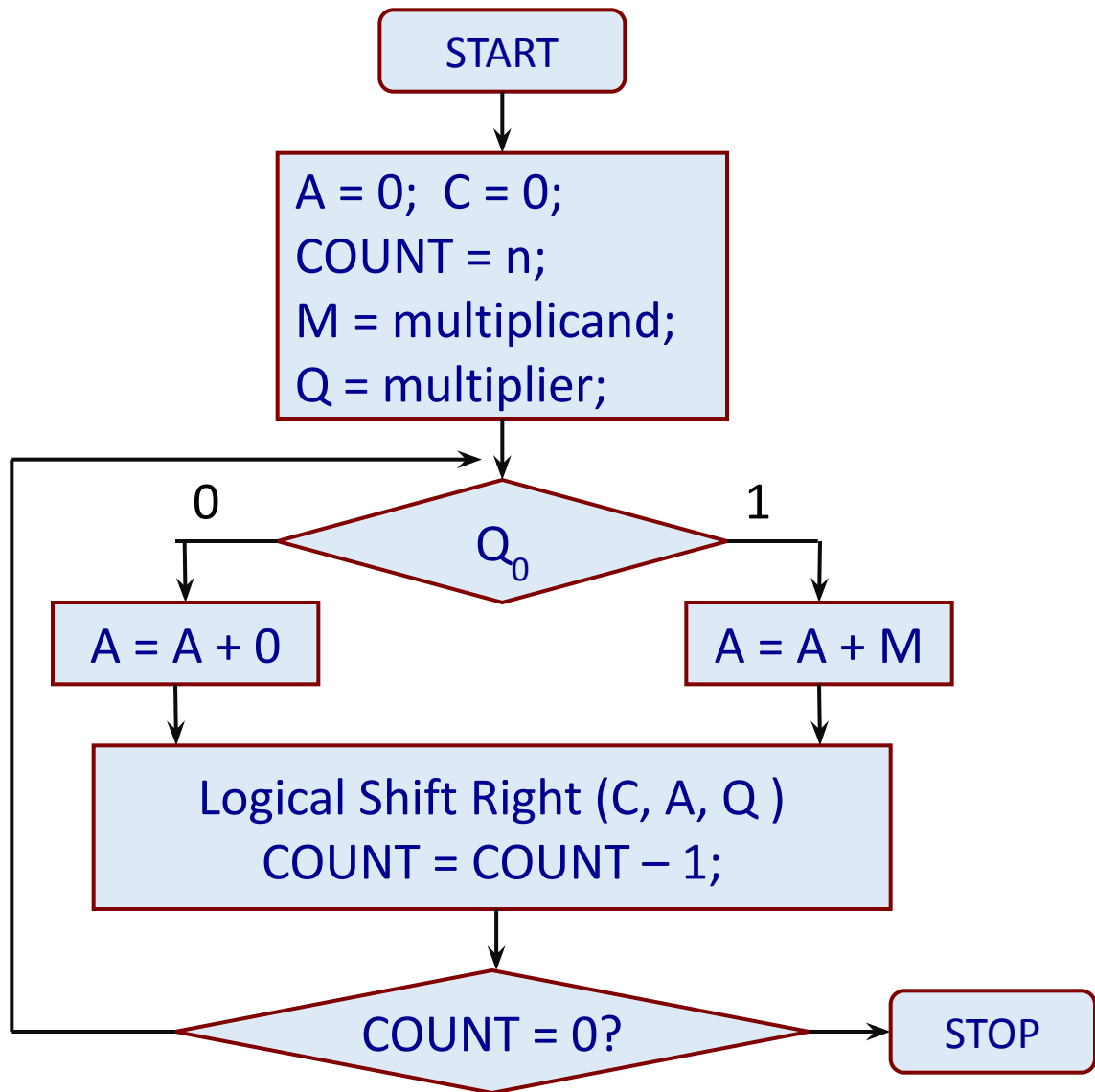


Product: p₇ p₆ p₅ p₄ p₃ p₂ p₁ p₀

Unsigned Sequential Multiplication

- Requires much less hardware, but requires several clock cycles to perform multiplication of two n -bit numbers.
 - Typical hardware complexity: $O(n)$
 - Typical time complexity: $O(n)$
- In the “*hand multiplication*” that we have seen:
 - If the i -th bit of the multiplier is 1 , the multiplicand is shifted left by i bit positions, and added to the partial product.
 - The relative position of the partial products do not change; it is the multiplicand that gets shifted left.

- In the “*shift-and-add*” multiplication that we discuss now, we make the following modifications.
 - We do not shift the multiplicand (i.e., keep its position fixed).
 - We right shift an $2n$ -bit partial product at every step.



M: n-bit multiplicand

Q: n-bit multiplier

A: n-bit temporary register

C: 1-bit carry out from adder

Example 1: $(10) \times (13)$

Assume 5-bit numbers.

M: $(0\ 1\ 0\ 1\ 0)_2$

Q: $(0\ 1\ 1\ 0\ 1)_2$

Product = 130

$= (0\ 0\ 1\ 0\ 0\ 0\ 0\ 0\ 1\ 0)_2$

C	A	Q		
0	0 0 0 0 0	0 1 1 0 1	Initialization	
0	0 1 0 1 0	0 1 1 0 1	A = A + M	Step 1
0	0 0 1 0 1	0 0 1 1 0	Shift	
0	0 0 1 0 1	0 0 1 1 0	A = A + 0	Step 2
0	0 0 0 1 0	1 0 0 1 1	Shift	
0	0 1 1 0 0	1 0 0 1 1	A = A + M	Step 3
0	0 0 1 1 0	0 1 0 0 1	Shift	
0	1 0 0 0 0	0 1 0 0 1	A = A + M	Step 4
0	0 1 0 0 0	0 0 1 0 0	Shift	
0	0 1 0 0 0	0 0 1 0 0	A = A + 0	Step 5
0	0 0 1 0 0	0 0 0 1 0	Shift	

Example 2: $(29) \times (21)$

Assume 5-bit numbers.

M: $(1\ 1\ 1\ 0\ 1)_2$

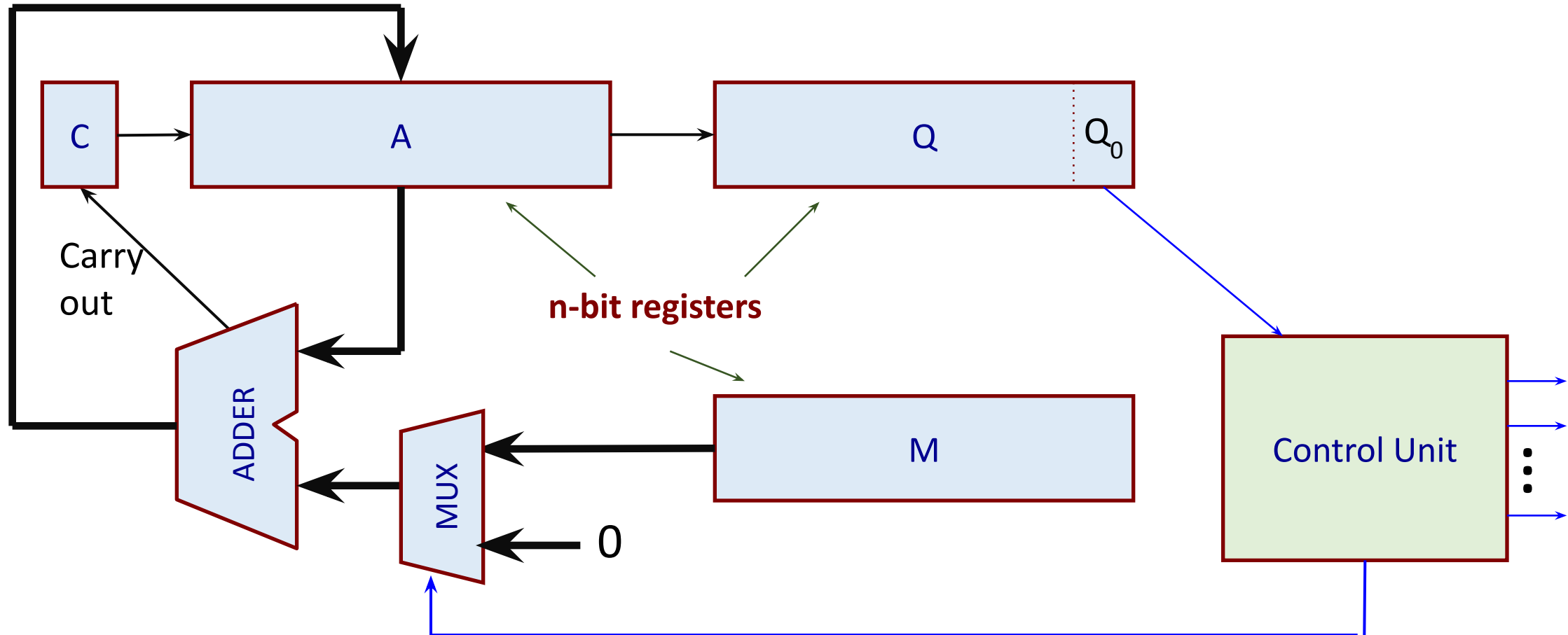
Q: $(1\ 0\ 1\ 0\ 1)_2$

Product = 609

$= (1\ 0\ 0\ 1\ 1\ 0\ 0\ 0\ 0\ 1)_2$

C	A	Q		
0	0 0 0 0 0	1 0 1 0 1	Initialization	
0	1 1 1 0 1	1 0 1 0 1	A = A + M	Step 1
0	0 1 1 1 0	1 1 0 1 0	Shift	
0	0 1 1 1 0	1 1 0 1 0	A = A + 0	Step 2
0	0 0 1 1 1	0 1 1 0 1	Shift	
1	0 0 1 0 0	0 1 1 0 1	A = A + M	Step 3
0	1 0 0 1 0	0 0 1 1 0	Shift	
0	1 0 0 1 0	0 0 1 1 0	A = A + 0	Step 4
0	0 1 0 0 1	0 0 0 1 1	Shift	
1	0 0 1 1 0	0 0 0 1 1	A = A + M	Step 5
0	1 0 0 1 1	0 0 0 0 1	Shift	

Data Path for Shift-and-Add Multiplier



Signed Multiplication

- We can extend the basic shift-and-add multiplication method to handle signed numbers.
- One important difference:
 - Require to sign-extend all the partial products before they are added.
 - Recall that for 2's complement representation, sign extension can be done by replicating the sign bit any number of times.

0101 = 0000 0101 = 0000 0000 0000 0101 = 0000 0000 0000 0000 0000 0000 0000 0101

1011 = 1111 1011 = 1111 1111 1111 1011 = 1111 1111 1111 1111 1111 1111 1111 1011

An Example: 6-bit 2's complement multiplication

Note: For n -bit multiplication, since we are generating a $2n$ -bit product, overflow can never occur.

```

      1 1 0 1 0 1      (-11)
    x 0 1 1 0 1 0      (+26)
    -----
0  0 0 0 0 0 0 0 0 0 0 0
1  1 1 1 1 1 1 1 0 1 0 1
0  0 0 0 0 0 0 0 0 0 0
1  1 1 1 1 1 0 1 0 1
1  1 1 1 0 1 0 1
0  0 0 0 0 0 0
    -----
1  1 1 0 1 1 1 0 0 0 1 0      (-286)

```

Booth's Algorithm for Signed Multiplication

- In the conventional shift-and-add multiplication as discussed, for n -bit multiplication, we iterate n times.
 - Add either 0 or the multiplicand to the $2n$ -bit partial product (depending on the next bit of the multiplier).
 - Shift the $2n$ -bit partial product to the right.
- Essentially we need n *additions* and n *shift operations*.
- Booth's algorithm is an improvement whereby we can avoid the additions whenever consecutive 0's or 1's are detected in the multiplier.
 - Makes the process faster.

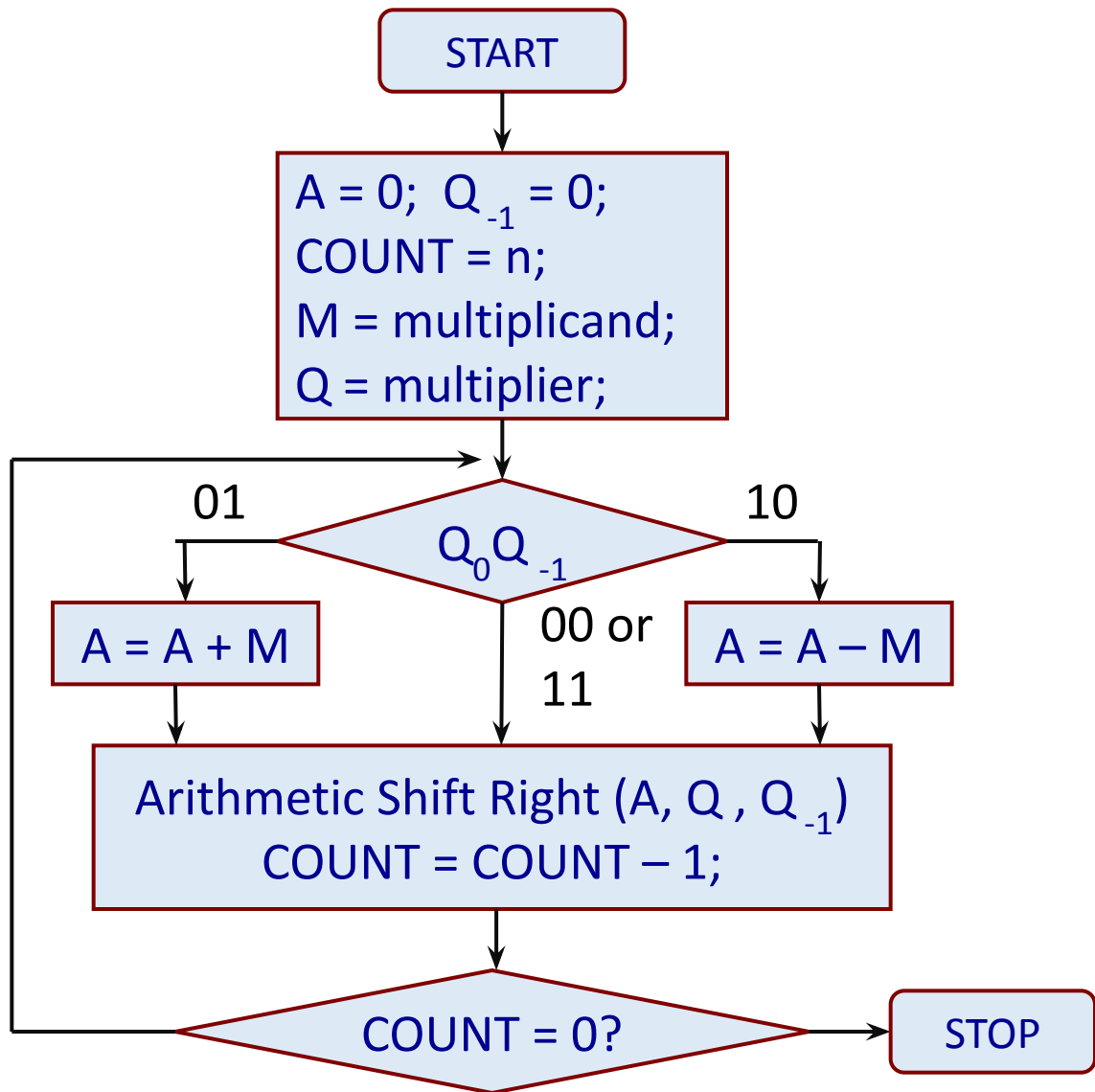
Basic Idea Behind Booth's Algorithm

- We inspect two bits of the multiplier (Q_i, Q_{i-1}) at a time.
 - If the bits are same (00 or 11), we only shift the partial product.
 - If the bits are 01, we do an addition and then shift.
 - If the bits are 10, we do a subtraction and then shift.
- Significantly reduces the number of additions / subtractions.
- Inspecting bit pairs as mentioned can also be expressed in terms of *Booth's Encoding*.
 - Use the symbols +1, -1 and 0 to indicate changes w.r.t. Q_i and Q_{i-1} .
 - 01 $\square$ +1, 10 $\square$ -1, 00 or 11 $\square$ 0.
 - For encoding the least significant bit Q_0 , we assume $Q_{-1} = 0$.

- Examples of Booth encoding:

- a) 0 1 1 1 0 0 0 0 :: +1 0 0 -1 0 0 0 0
- b) 0 1 1 1 0 1 1 0 :: +1 0 0 -1 +1 0 -1 0
- c) 0 0 0 0 0 1 1 1 :: 0 0 0 0 +1 0 0 -1
- d) 0 1 0 1 0 1 0 1 :: +1 -1 +1 -1 +1 -1 +1 -1

- The last example illustrates the *worst case* for Booth's multiplication (alternating 0's and 1's in multiplier).
 - In the illustrations, we shall show the two multiplier bits explicitly instead of showing the encoded digits.



M: n-bit multiplicand

Q: n-bit multiplier

A: n-bit temporary register

Q_{-1} : 1-bit flip-flop

**Skips over consecutive 0's
and 1's of the multiplier Q**

Example 1: $(-10) \times (13)$

Assume 5-bit numbers.

M: $(1\ 0\ 1\ 1\ 0)_2$

-M: $(0\ 1\ 0\ 1\ 0)_2$

Q: $(0\ 1\ 1\ 0\ 1)_2$

Product = -130

$= (1\ 1\ 0\ 1\ 1\ 1\ 1\ 1\ 0)_2$

A	Q	Q ₋₁	
0 0 0 0 0	0 1 1 0	1 0	Initialization

0 1 0 1 0	0 1 1 0	1 0	A = A - M
0 0 1 0 1	0 0 1 1	0 1	Shift

Step 1

1 1 0 1 1	0 0 1 1	0 1	A = A + M
1 1 1 0 1	1 0 0 1	1 0	Shift

Step 2

0 0 1 1 1	1 0 0 1	1 0	A = A - M
0 0 0 1 1	1 1 0 0	1 1	Shift

Step 3

0 0 0 0 1	1 1 1 1	0 1	Shift
-----------	---------	-----	-------

Step 4

1 0 1 1 1	1 1 1 0 0	1	A = A + M
1 1 0 1 1	1 1 1 1 0	0	Shift

Step 5

Example 2:

$(-31) \times (28)$

Assume 6-bit
numbers.

M: $(1\ 0\ 0\ 0\ 0\ 1)_2$

-M: $(0\ 1\ 1\ 1\ 1\ 1)_2$

Q: $(0\ 1\ 1\ 1\ 0\ 0)_2$

Product = -868

= $(1\ 1\ 0\ 0\ 1\ 0$

$0\ 1\ 1\ 1\ 0\ 0)_2$

A	Q	Q ₋₁	
0 0 0 0 0 0	0 1 1 1 0	0 0	Initialization
0 0 0 0 0 0	0 0 1 1 1	0 0	Shift
0 0 0 0 0 0	0 0 0 1 1	1 0	Shift
0 1 1 1 1 1	0 0 0 1 1	1 0	A = A - M
0 0 1 1 1 1	1 0 0 0 1	1 1	Shift
0 0 0 1 1 1	1 1 0 0 0	1 1	Shift
0 0 0 0 1 1	1 1 1 0 0	0 1	Shift
1 0 0 1 0 0	1 1 1 0 0	1 1	A = A + M
1 1 0 0 1 0	0 1 1 1 0 0	0 0	Shift

Step 1

Step 2

Step 3

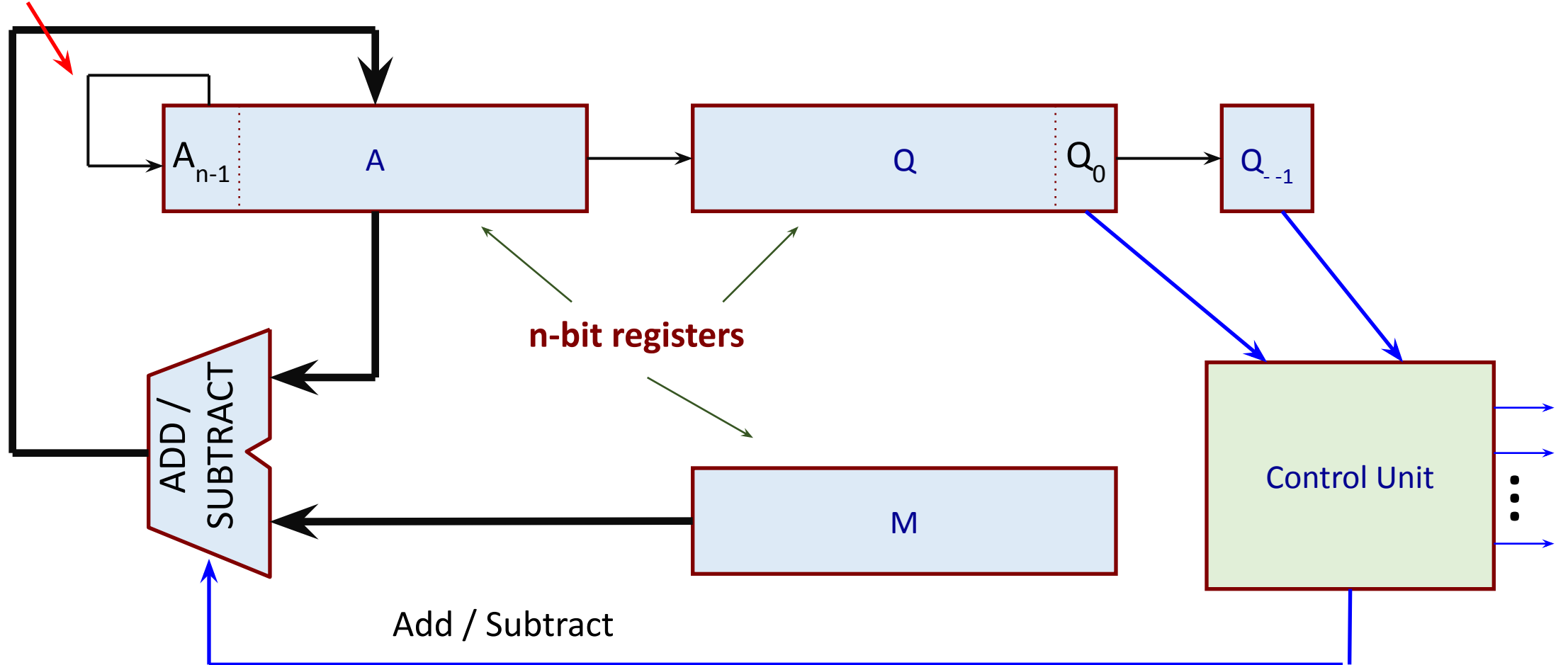
Step 4

Step 5

Step 6

Arithmetic
shift right

Data Path for Booth's Algorithm



Design of Fast Multiplier

a) Bit-Pair Recoding of Booth's Multiplication

- A technique that halves the maximum number of summands; derived directly from the Booth's algorithm.
- If we group the Booth-coded multiplier digits in pairs, we observe:
 - (+1, -1): $(+1, -1) * M = 2 * M - M = M$
 - (0, +1): $(0, +1) * M = M$
- We need a single addition instead of a pair of addition & subtraction.
 - Other similar rules can be framed.
 - Shown on next slide.

Original Booth-coded Pair	Equivalent Recoded Pair
(+1, 0)	(0, +2)
(-1, +1)	(0, -1)
(0, 0)	(0, 0)
(0, 1)	(0, 1)
(+1, 1)	--
(+1, -1)	(0, +1)
(-1, 0)	(0, -2)

- Every equivalent recoded pair has at least one 0.
- Worst-case number of additions or subtractions is 50% of the number of multiplier bits.
- Reduces the worst-case time required for multiplication.

Example: (+13) X (-22) in 6-bits.

Original:	Multiplier	--	1	0	1	0	1	0
Booth:	Multiplier	--	-1	+1	-1	+1	-1	0
Recoded:	Multiplier	--	0	-1	0	-1	0	-2

			0	0	1	1	0	1	
	.	-1	.	-1	.	-2			

1	1	1	1	1	1	1	0	0	1
1	1	1	1	1	1	0	0	1	1
1	1	1	1	0	0	1	1		

1	1	0	1	1	1	1	0	0	0

- M = 001101 (+13)
- -1 * M = 110011
- -2 * M = 100110

Carry Save Adder

- Here we add three operands (say, X , Y and Z) together.
- For adding multiple numbers, we have to construct a tree of carry save adders.
 - Used in combinational multiplier design.
- Each carry save adder is simply an independent full adder without carry propagation.
- A parallel adder is required only at the last stage.

- **An illustrative example:**

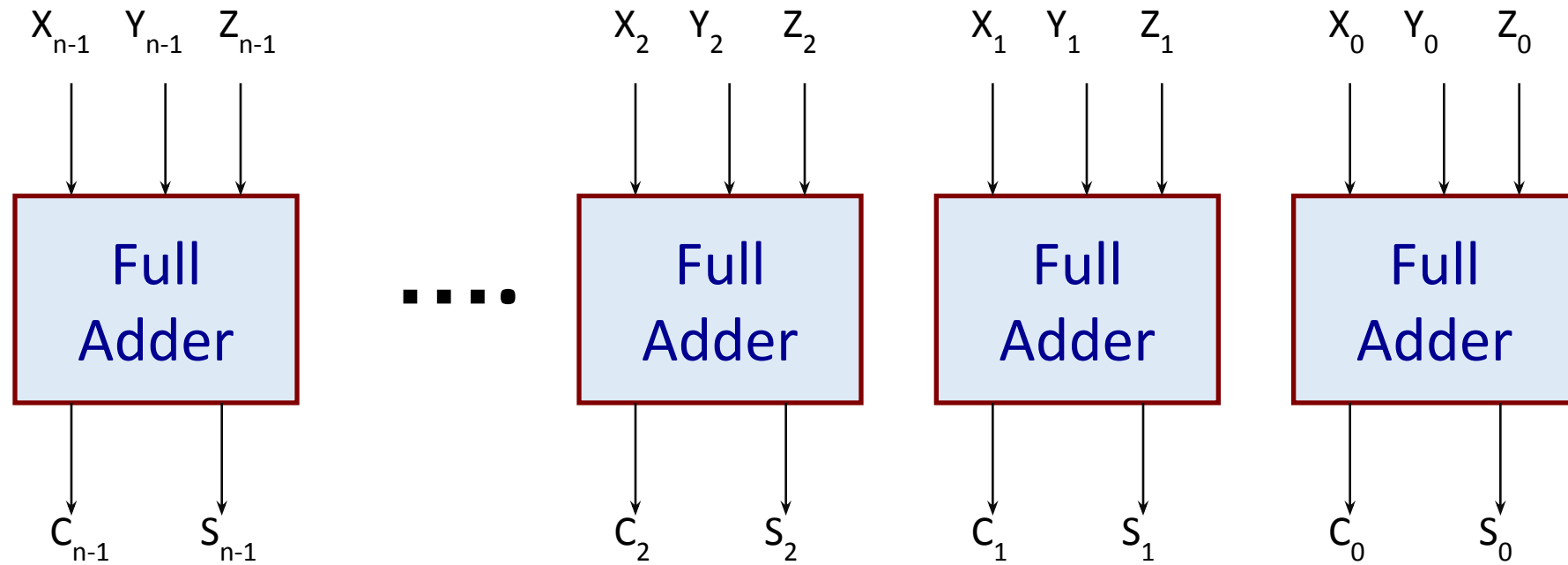
X:	1 0 0 1 1	X:	1 0 0 1 1
Y:	+ 1 1 0 0 1	Y:	+ 1 1 0 0 1
Z:	+ 0 1 0 1 1	Z:	+ 0 1 0 1 1
C:	1 1 0 1 1	S:	0 0 0 0 1

A set of full adders generate carry
and sum bits in parallel

X:	1 0 0 1 1
Y:	+ 1 1 0 0 1
Z:	+ 0 1 0 1 1
S:	0 0 0 0 1
C:	1 1 0 1 1
Sum:	1 1 0 1 1 1

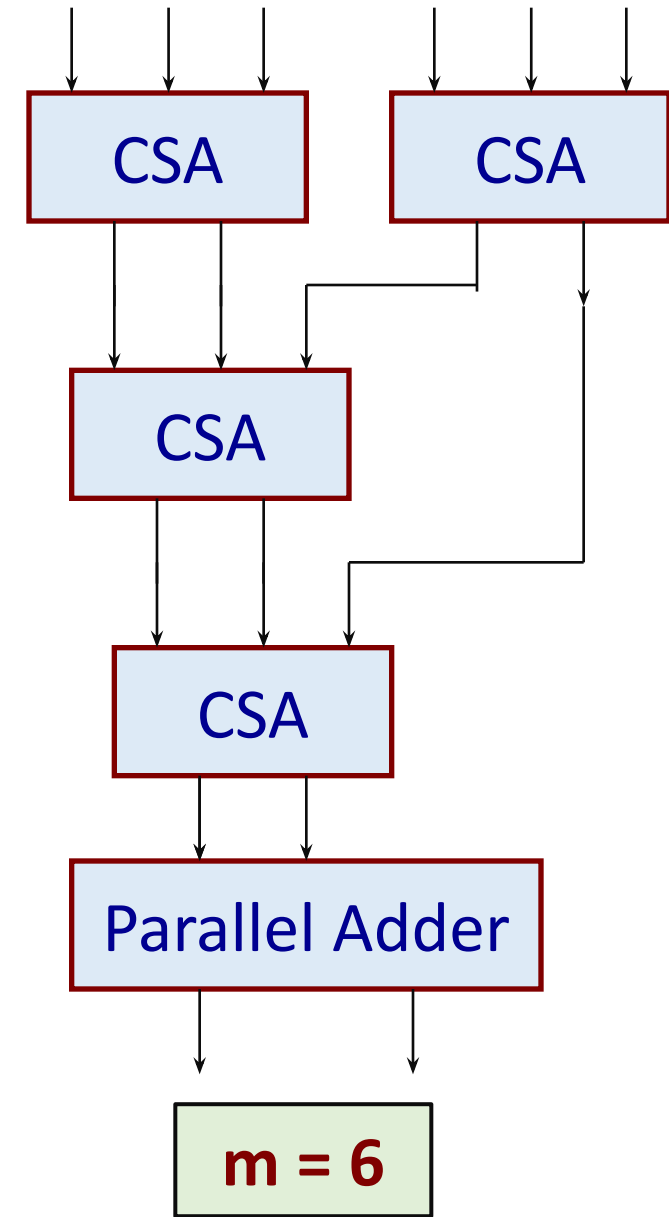
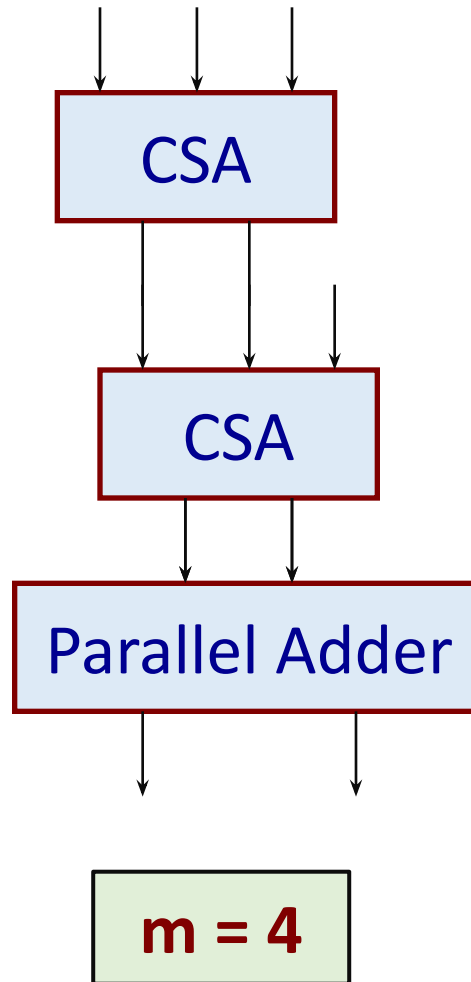
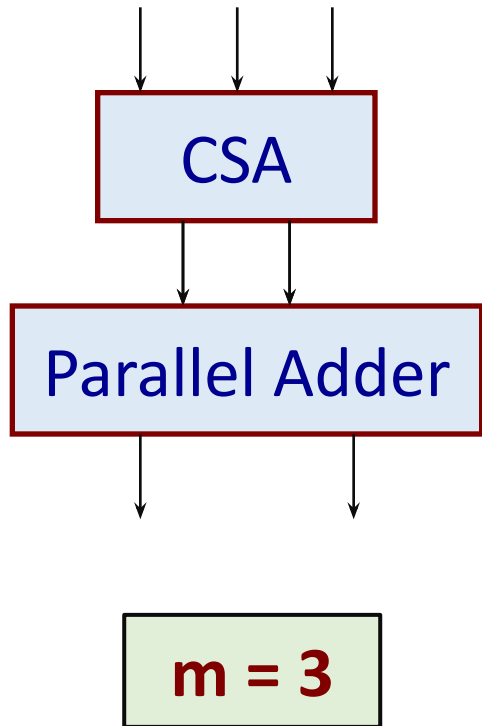
The sum and carry vectors are
added later (with proper shifting)

An n-bit Carry Save Adder



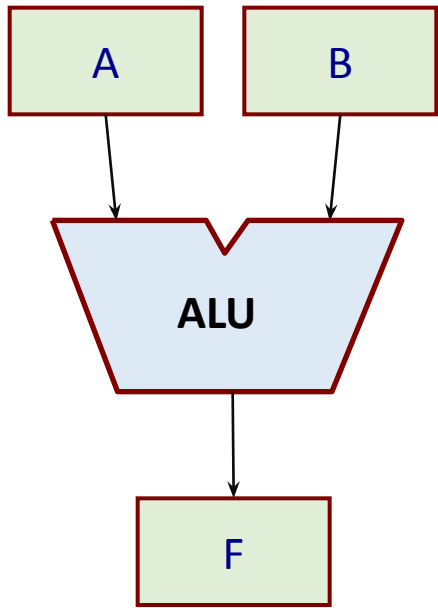
The carry input of the full adder is used as the third input

Adding m Numbers: Some Examples

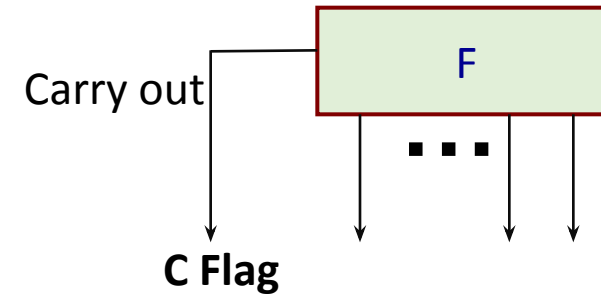
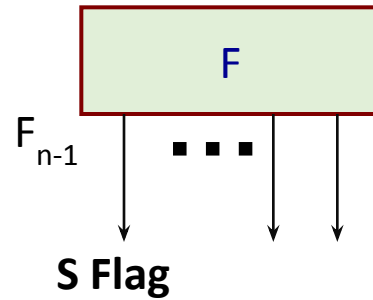
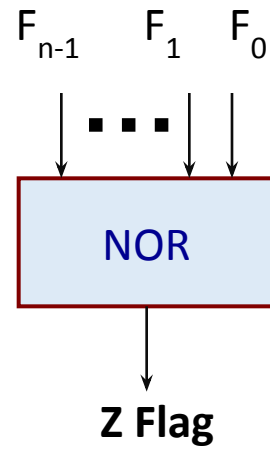


Generating the Status Flags

- Many contemporary processors have a *flag register* that contains the status of the last arithmetic / logic operation.
 - **Zero (Z)**: tells whether the result is zero.
 - Can be used for both arithmetic and logic operations.
 - **Sign (S)**: tells whether the result is positive (=0) or negative (=1).
 - Can be used for both arithmetic and logic operations.
 - **Carry (C)**: tells whether there has been a carry out of the most significant stage.
 - Used only for arithmetic operations.
 - **Overflow (V)**: tells whether the result is too large to fit in the target register.
 - Used only for arithmetic operations (addition and subtraction).



Assume A , B and F are n -bit registers



- Overflow can occur during addition when the sign of the two operands are the same.
 - Sign of the result becomes different from the sign of the operand(s).

$$V = A_{n-1} \cdot B_{n-1} \cdot F_{n-1}' + A_{n-1}' \cdot B_{n-1}' \cdot F_{n-1}$$

$$V = F_{n-1} \oplus \text{Carry_out}$$

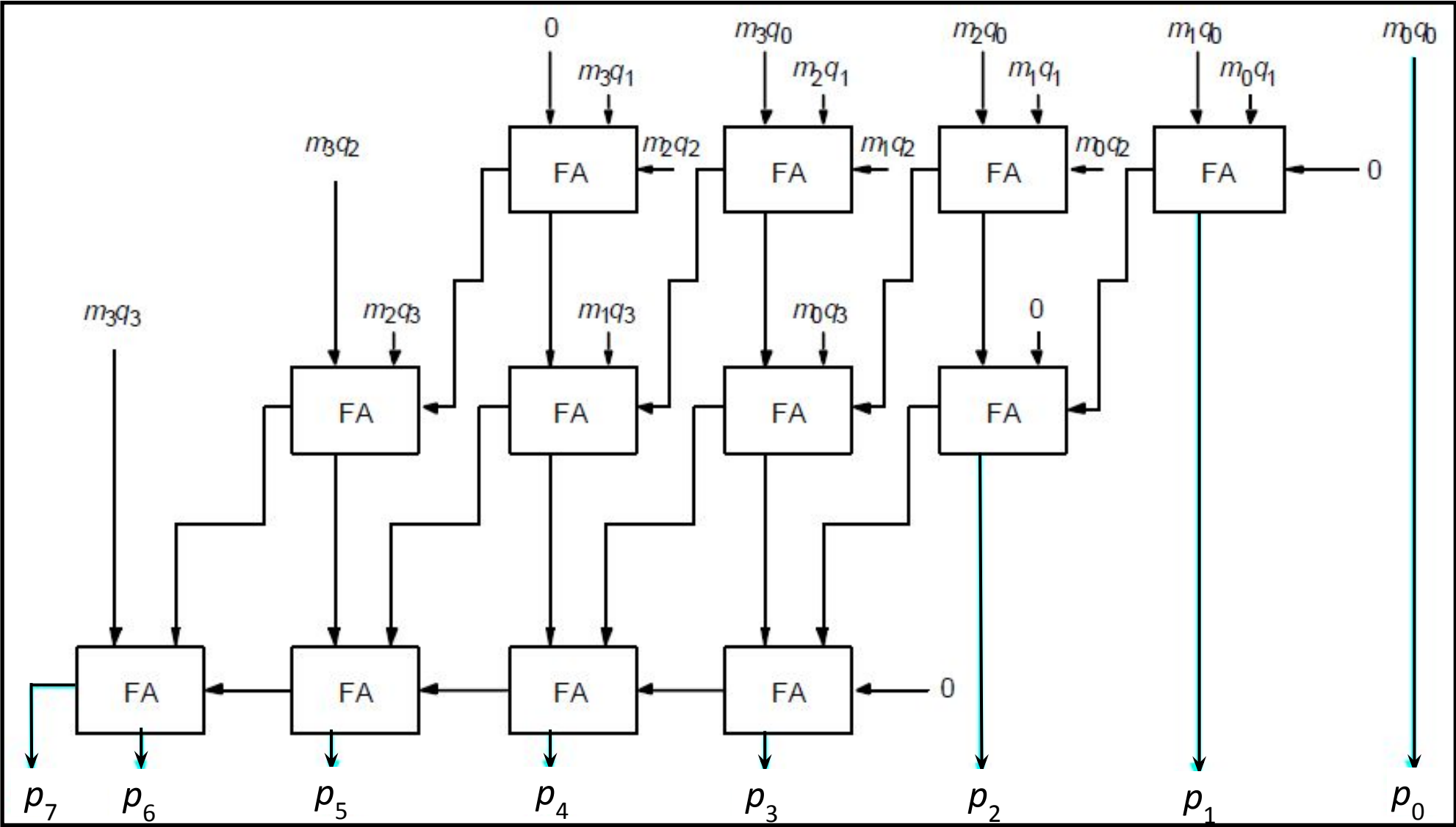
- The MIPS32 processor does not have any status flags.
- Why?
 - MIPS ISA is designed for efficient pipeline implementation.
 - Several instructions can be in various stages of execution in the pipeline.
 - Flag registers result in *side effects* among instructions.
- MIPS stores information about the flags temporarily in a GPR.

```
slt$t0, $s1, $s2  
beq  $t0, $zero, Label
```

b) Carry Save Multiplier

- We have seen earlier how carry save adders (CSA) can be used to add several numbers with carry propagation only in the last stage.
- The partial products can be generated in parallel using n^2 AND gates.
- The n partial products can then be added using a CSA tree.
- Instead of letting the carries ripple through during addition, we *save* them and feed it to the next row, at the correct weight positions.

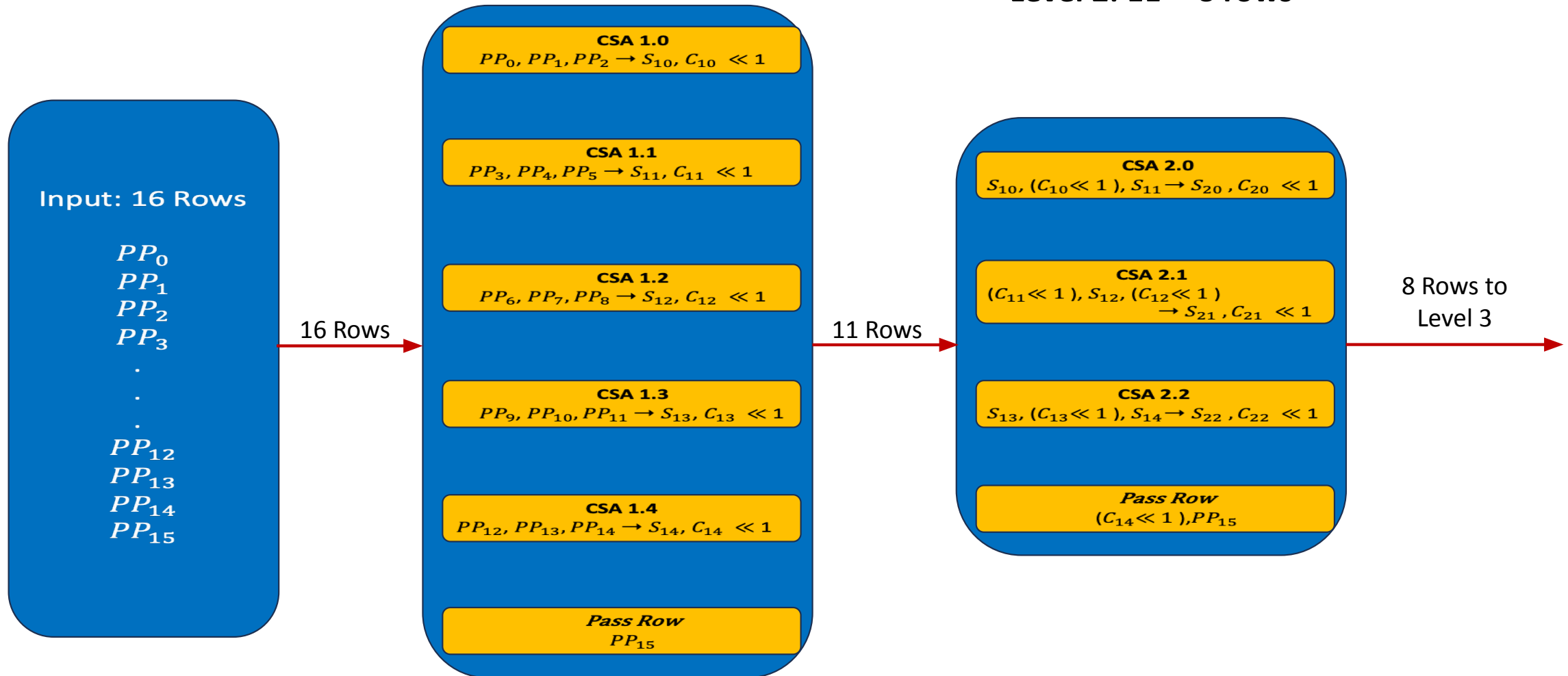
4 x 4 Carry Save Multiplier



Carry-Save Reduction Tree (Levels 1 and 2)

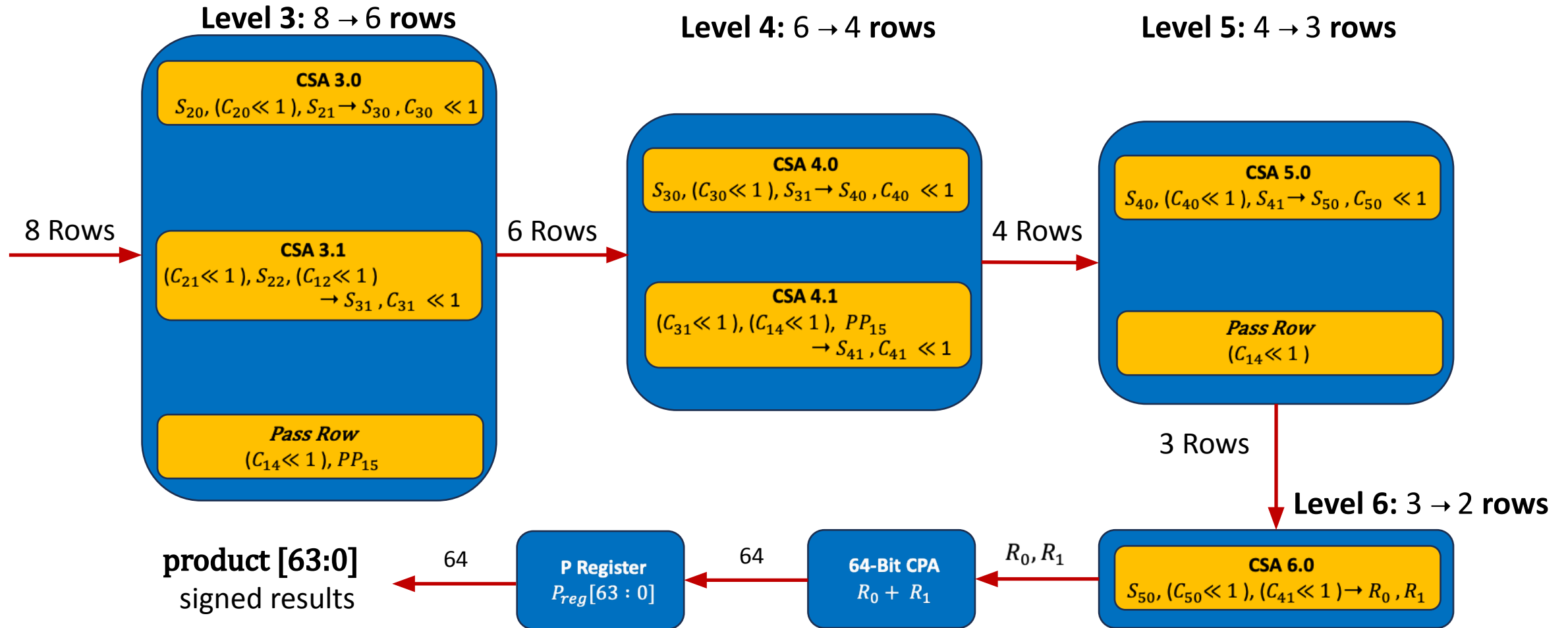
Level 1: 16 → 11 rows

Level 2: 11 → 8 rows



Every CSA is a 64-bit bank of independent full adders. For one CSA, $X + Y + Z = S + (C \ll 1)$. The carry vector is saved as a separate row at the next higher bit weight; it does not ripple horizontally inside the CSA. The boxes above list the exact rows consumed and produced at each compressor.

Carry-Save Reduction Tree (Levels 3 to 6 and Final Addition)

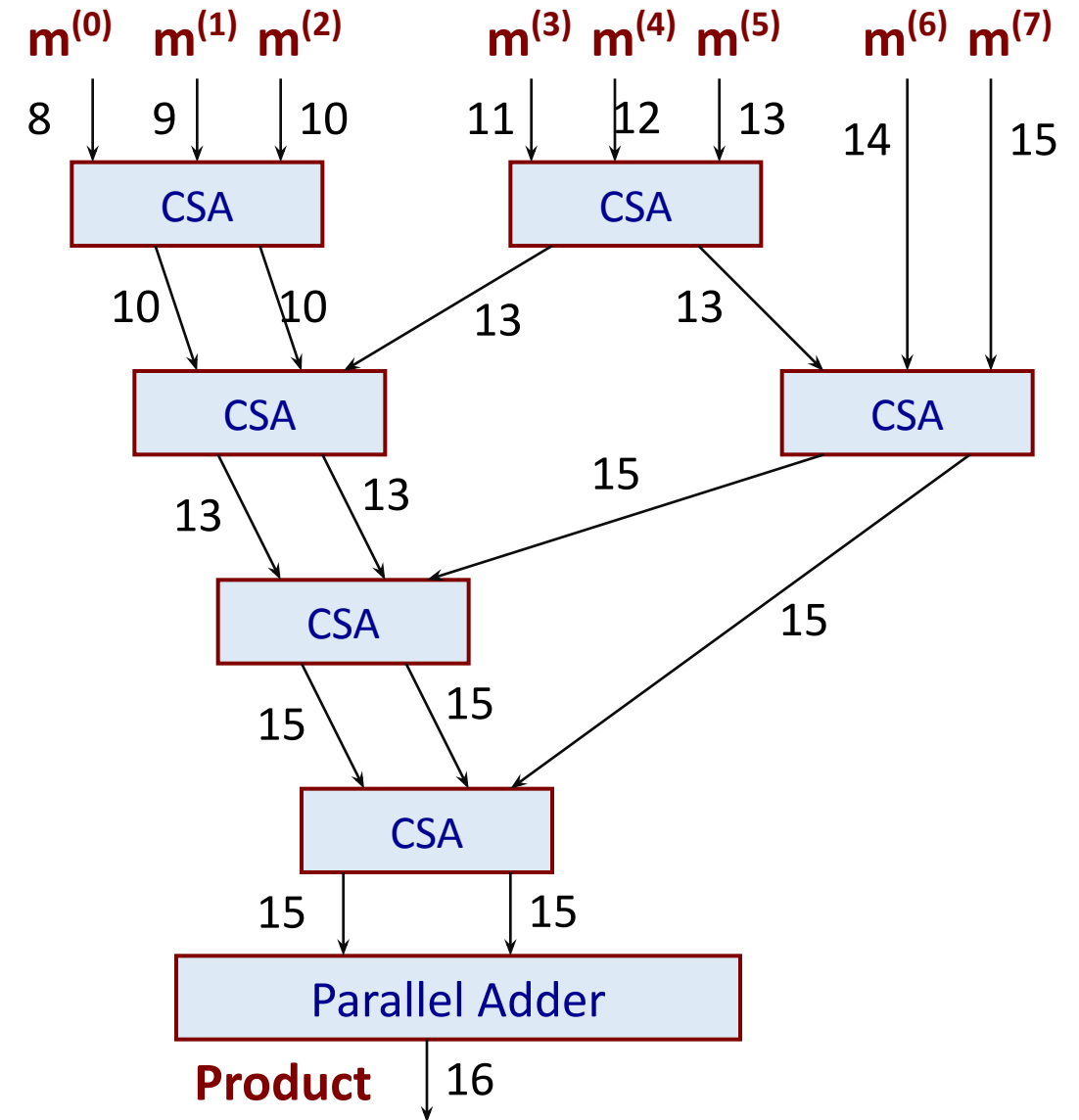


Only the final carry-propagate adder propagates carry across bit positions. Levels 1–6 use 3:2 carry-save compression. The stage boxes identify the exact rows grouped at every reduction level, while the thick arrows show the aggregate row bundles passed to the next stage.

- **Wallace Tree Multiplier**

- A Wallace tree is a circuit that reduces the problem of summing n n -bit numbers to the problem of summing two $\Theta(n)$ -bit numbers.
- It uses $n/3$ (floor of) carry-save adders in parallel to convert the sum of n numbers to the sum of $2n/3$ (ceiling of) numbers.
- It then recursively constructs a Wallace tree on the $2n/3$ (ceiling of) resulting numbers.
- The set of numbers is progressively reduced until there are only two numbers left.
- By performing many carry-save additions in parallel, Wallace trees allow two n -bit numbers to be multiplied in $\Theta(\log_2 n)$ time using a circuit of size $\Theta(n^2)$.

- The figure shows a Wallace tree that adds 8 partial products $m^{(0)}, m^{(1)}, \dots, m^{(7)}$.
- The partial product $m^{(i)}$ consists of $(n + i)$ bits.
- Each line represents an entire number – the label of an edge indicates the number of bits.
- The carry-lookahead adder at the bottom adds a $(2n-1)$ -bit number to a $2n$ -bit number to give the $2n$ -bit product.



Division

Introduction

- Division is more complex than multiplication.
- Example: Typical values in Pentium-3 processor ☐
 - Not easy to construct high-speed dividers.
- The ratios have not changed much in later processors.

Instruction	Latency	Cycles / Issue
Load / Store	3	1
Integer Multiply	4	1
Integer Divide	36	36
Floating-point Add	3	1
Floating-point Multiply	5	2
Floating-point Divide	38	38

- **Latency:**

- Minimum delay after which the first result is obtained, starting from the time when the first set of inputs is applied.

- **Cycles/Issue:**

- Whenever a new set of inputs is applied to a functional unit (e.g. adder), it is called an *issue*.
- Pipelined implementation of arithmetic unit reduces the number of clock cycles between successive issues.
- For non-pipelined arithmetic units (e.g. divider), the number of clock cycles between successive issues is much higher.
 - Next input can be applied only after the previous operation is complete.

The Process of Integer Division

- In integer division, a *divisor* M and a *dividend* D are given.
- The objective is to find a third number Q , called the *quotient*, such that

$$D = Q \times M + R$$

where R is the *remainder* such that $0 \leq R < M$.

- The relationship $D = Q \times M$ suggests that there is a close correspondence between division and multiplication.
 - Dividend, quotient and divisor correspond to product, multiplicand and multiplier, respectively.
 - Similar algorithms and circuits can be used for multiplication and division.

- One of the simplest division methods is the sequential digit-by-digit algorithm similar to that used in pencil-and-paper methods.

$$D = 37 = (1\ 0\ 0\ 1\ 0\ 1)_2$$

$$M = 6 = (1\ 1\ 0)_2$$

Quotient $Q = 6$

Remainder $R = 1$

	0	1	1	0		Quotient $Q = Q_0Q_1Q_2Q_3$
1	0	0	1	0	1	Dividend $D = R_0$
1	1	0				$Q_0 \cdot M$ (Does not go; $Q_0 = 0$)

1	0	0	1	0	1	R_1
-	1	1	0			$Q_1 \cdot 2^{-1} \cdot M$ (Does go; $Q_1 = 1$)

	0	1	1	0	1	R_2
-		1	1	0		$Q_2 \cdot 2^{-2} \cdot M$ (Does go; $Q_2 = 1$)

	0	0	0	1		R_3
	1	1	0			$Q_3 \cdot 2^{-3} \cdot M$ (Does not go; $Q_3 = 0$)

	0	0	1			$R_4 = \text{Remainder } R$

- In the example, the quotient $Q = Q_0Q_1Q_2\dots$ is computed one bit at a time.
 - At each step i , the divisor shifted i bits to the right (i.e. $2^{-i} \cdot M$) is compared with the current partial remainder R_i .
 - The quotient bit Q_i is set to 0 (1) if $2^{-i} \cdot M$ is greater than (less than) R_i .
 - The new partial remainder R_{i+1} is computed as:

$$R_{i+1} = R_i - Q_i \cdot 2^{-i} \cdot M$$

- **Machine implementation:**

- For hardware implementation, it is more convenient to shift the partial remainder to the left relative to a fixed divisor; thus

$$R_{i+1} = 2R_i - Q_i \cdot M \quad (\text{instead of } R_{i+1} = R_i - Q_i \cdot 2^{-i} \cdot M)$$

- The final partial remainder is the required remainder shifted to the left, so that $R = 2^{-3} \cdot R_4$ (see next slide).

Divisor M

1 1 0

1 0 0 1 0 1

Dividend = $2R_0$

Quotient Q

0

Do not
subtract

1 1 0 $Q_0 \cdot M$

1 0 0 1 0 1 R_1

1 0 0 1 0 1 0 $2R_1$

1 1 0 $Q_1 \cdot M$

0 1 1 0 1 0 R_2

0 1 1 0 1 0 0 $2R_2$

1 1 0 $Q_2 \cdot M$

0 0 0 1 0 0 R_3

0 0 0 1 0 0 0 $2R_3$

1 1 0 $Q_3 \cdot M$

0 0 1 0 0 0 $R_4 = 2^3 \cdot R$

0 1

0 1 1

0 1 1 0

$$D = 37 = (100101)_2$$

$$M = 6 = (110)_2$$

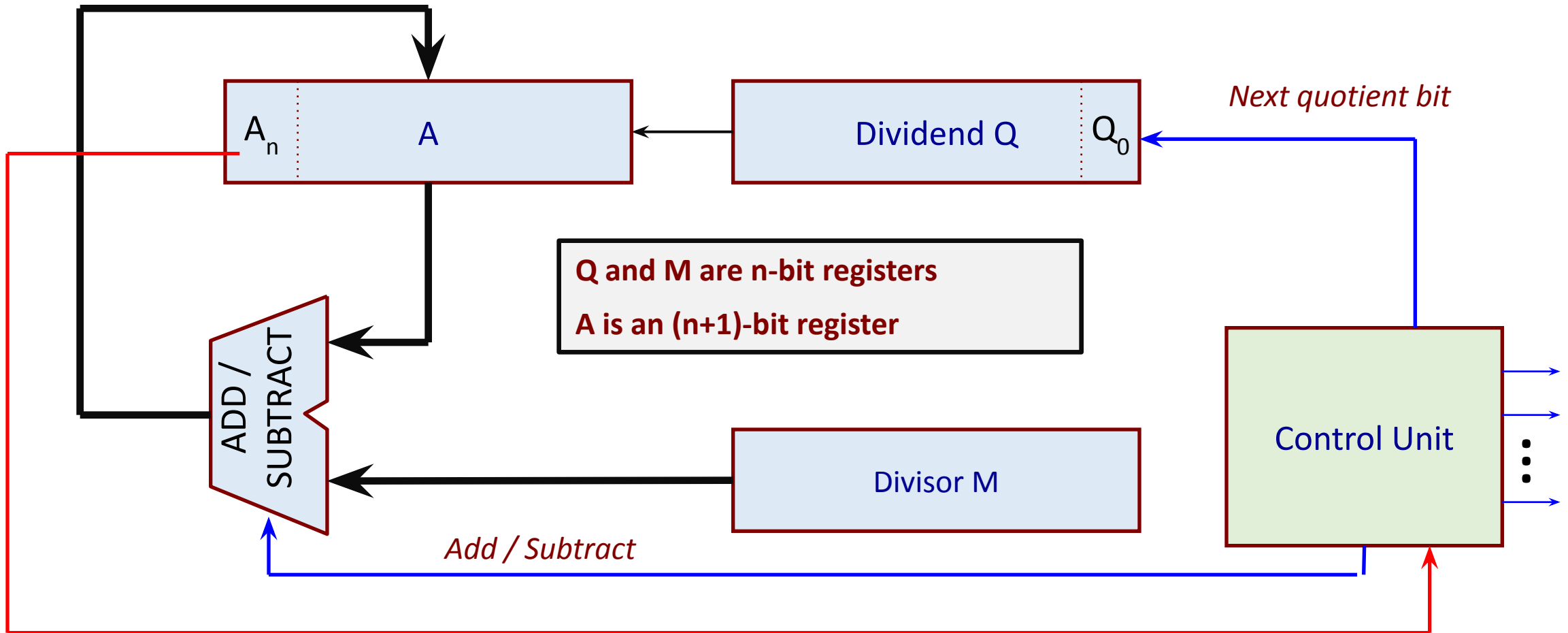
Quotient $Q = 6$

Remainder $R = 1$

Two alternatives to division

- We shall discuss two approaches:
 - a) Restoring division
 - b) Non-restoring division

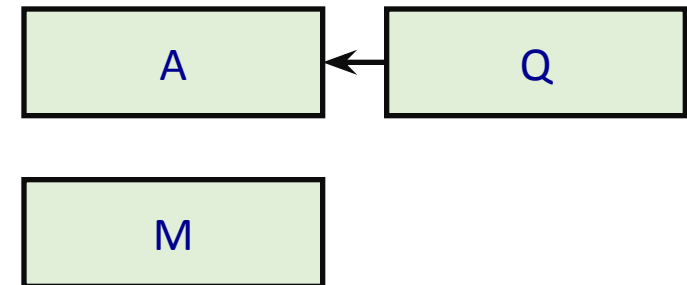
(a) Restoring Division: The Data Path

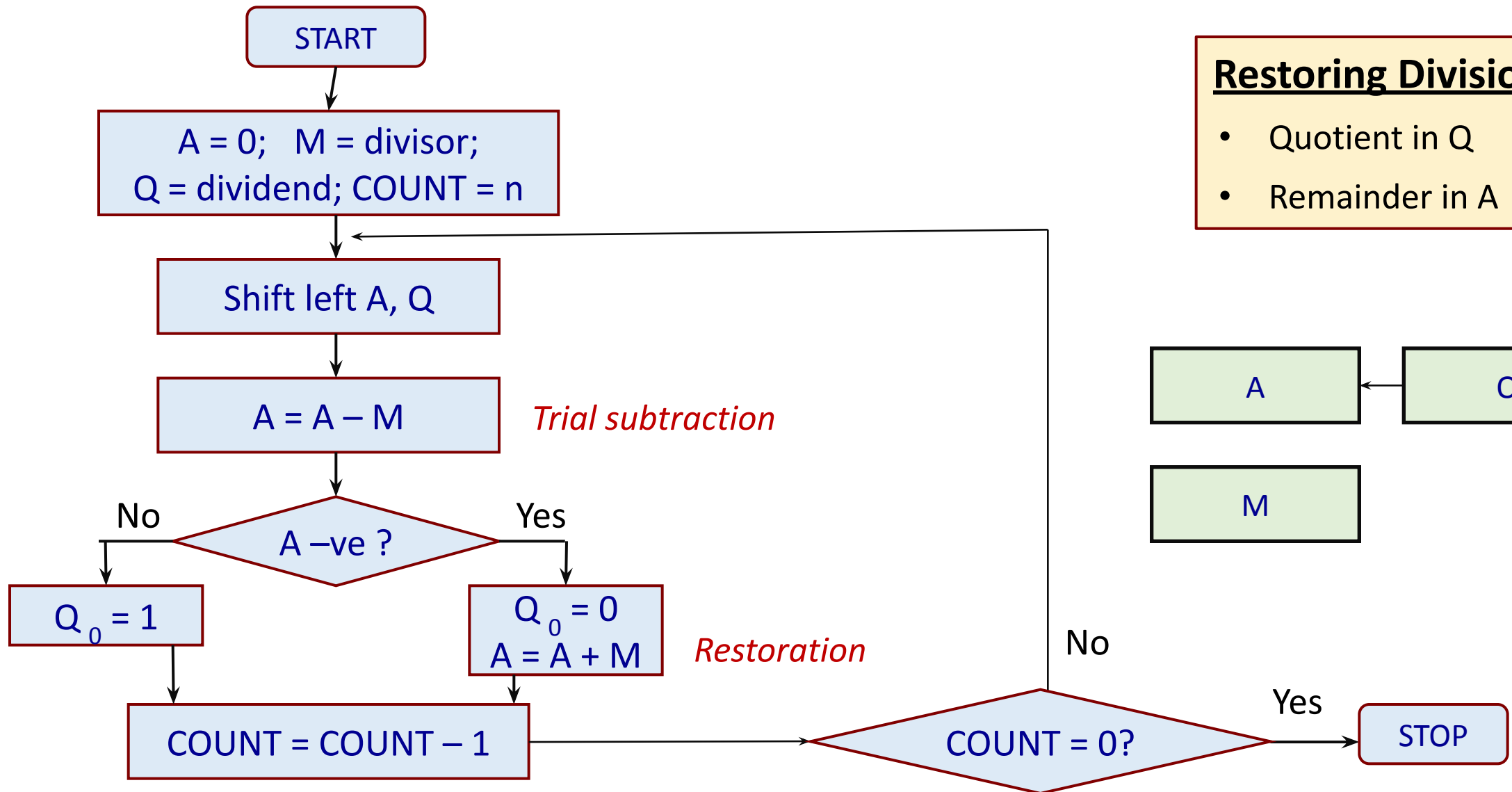


Basic Steps (Restoring Division)

Repeat the following steps n times:

- a) Shift the dividend left one bit at a time into register A .
- b) Subtract the divisor M from this register A (*trial subtraction*).
- c) If the result is negative (*i.e. not going*):
 - Add the divisor M back into the register A (*i.e. restoring back*).
 - Record 0 as the next quotient bit.
- d) If the result is positive:
 - Do not restore the intermediate result.
 - Record 1 as the next quotient bit.





- **Analysis:**

- For n -bit divisor and n -bit dividend, we iterate n times.
- Number of trial subtractions: n
- Number of restoring additions: $n/2$ on the average
 - Best case: 0
 - Worst case: n

A Simple Example: 8/3 for 4-bit representation (n=4)

Initially: 0 0 0 0 0 1 0 0 0

Shift: 0 0 0 0 1 0 0 0 -

Subtract: 0 0 1 1

Set Q_0 : (1) 1 1 1 0

Restore: 0 0 1 1

0 0 0 0 1 0 0 0 (0)

Shift: 0 0 0 1 0 0 0 0 -

Subtract: 0 0 1 1

Set Q_0 : (1) 1 1 1 1

Restore: 0 0 1 1

0 0 0 1 0 0 0 0 (0)

Shift: 0 0 1 0 0 0 0 0 -

Subtract: 0 0 1 1

Set Q_0 : (0) 0 0 0 1

0 0 0 0 0 0 0 0 (1)

Shift: 0 0 0 1 0 0 0 1 -

Subtract: 0 0 1 1

Set Q_0 : (1) 1 1 1 1

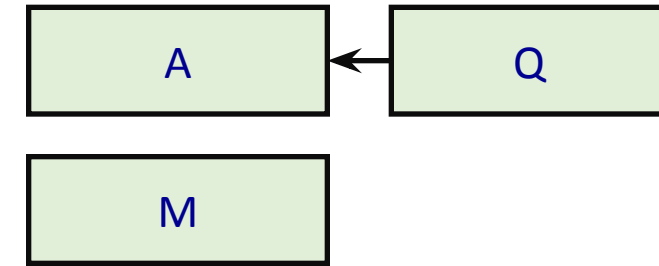
Restore: 0 0 1 1

0 0 0 1 0 0 0 1 (0)

Remainder
00010 = 2

Quotient
0010 = 2

(b) Non-Restoring Division



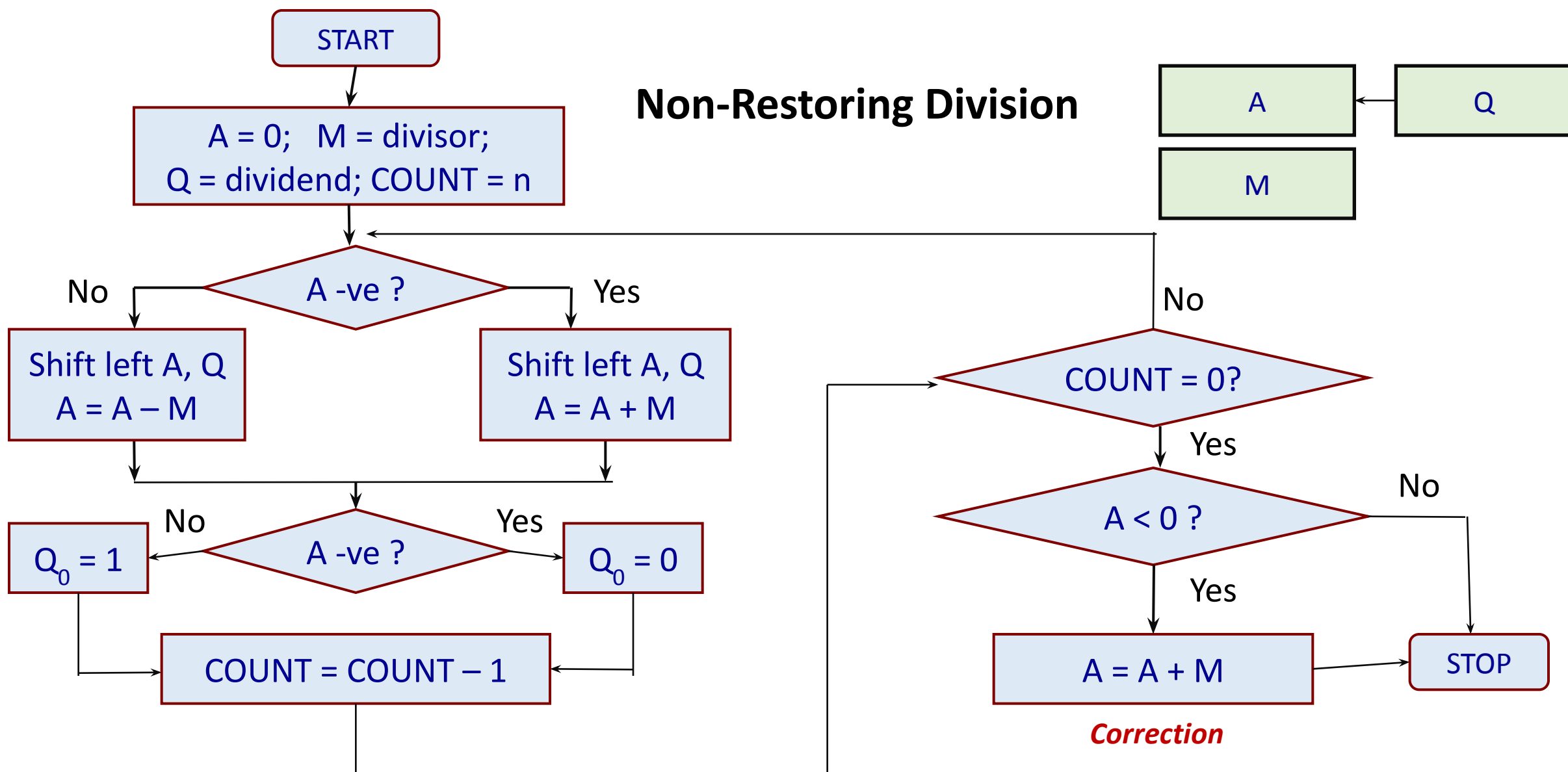
- The performance of restoring division algorithm can be improved by exploiting the following observation.
- In restoring division, what we do actually is:
 - If A is positive, we shift it left and subtract M .
 - That is, we compute $2A - M$.
 - If A is negative, we restore is by doing $A + M$, shift it left, and then subtract M .
 - That is, we compute $2(A + M) - M = 2A + M$.
- We can accordingly modify the basic division algorithm by eliminating the restoring step □ *NON-RESTORING DIVISION*.

Shift left means
multiplying by 2.

- **Basic steps in non-restoring division:**

- a) Start by initializing register A to 0, and repeat steps (b)-(d) n times.
- b) If the value in register A is positive,
 - Shift A and Q left by one bit position.
 - Subtract M from A .
- c) If the value in register A is negative,
 - Shift A and Q left by one bit position.
 - Add M to A .
- d) If A is positive, set $Q_o = 1$; else, set $Q_o = 0$.
- e) If A is negative, add M to A as a final corrective step.

Non-Restoring Division



A Simple Example: $8/3$ for $n=4$

Initially: 0 0 0 0 0 1 0 0 0

Shift: 0 0 0 0 1 0 0 0 -

Subtract: - 0 0 1 1

Set Q_0 : (1) 1 1 1 0 0 0 0 (0)

Shift: 1 1 1 0 0 0 0 0 -

Add: 0 0 1 1

Set Q_0 : (1) 1 1 1 1 0 0 0 (0)

Shift: 1 1 1 1 0 0 0 0 -

Add: 0 0 1 1

Set Q_0 : (0) 0 0 0 1 0 0 0 (1)

Shift: 0 0 0 1 0 0 0 1 -

Subtract: - 0 0 1 1

Set Q_0 : (1) 1 1 1 1 0 0 1 (0)

Correction Add:

1 1 1 1 1

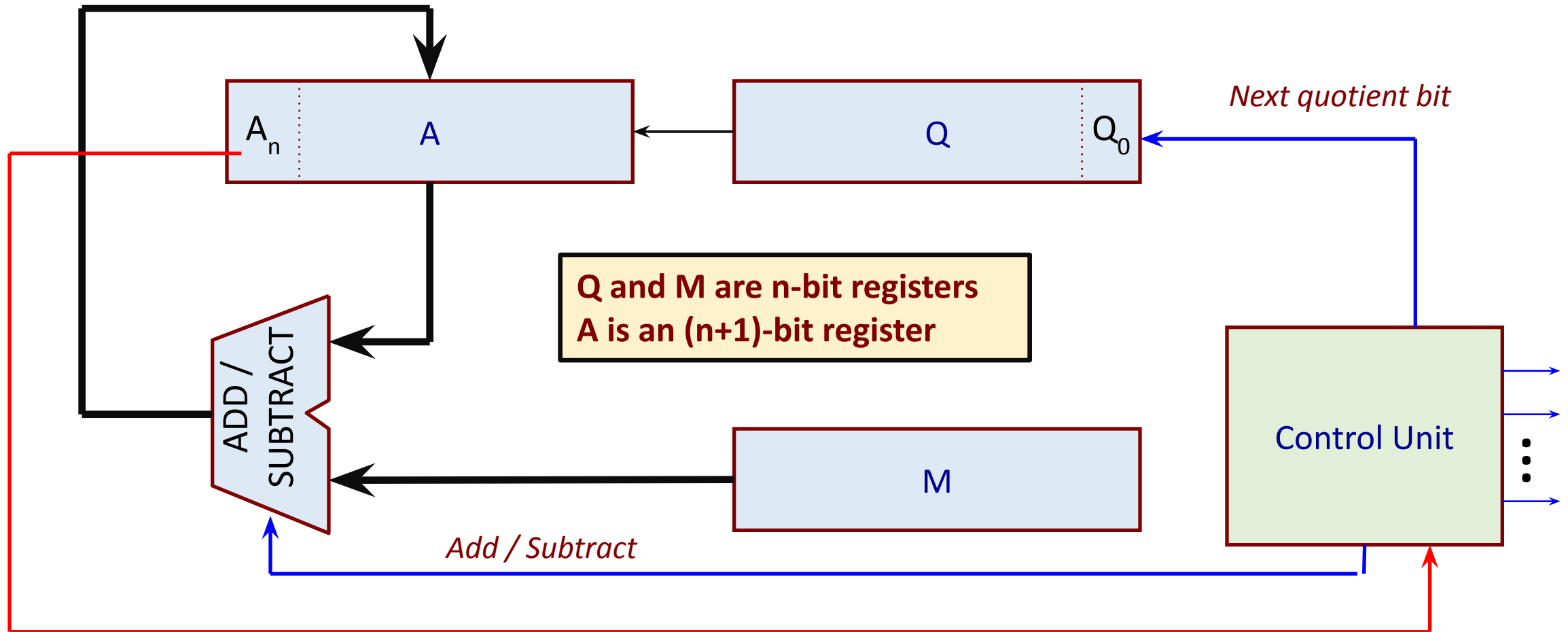
0 0 0 1 1

0 0 0 1 0

Quotient
0010 = 2

Remainder
00010 = 2

Data Path for Non-Restoring Division



High Speed Dividers

- Some of the methods used to increase the speed of multiplication can also be modified to speed up division.
 - High-speed addition and subtraction.
 - High-speed shifting.
 - Combinational array divider (implementing restoring division).
- The main difficulty is that it is very difficult to implement division in a pipeline to improve the performance.
 - Unlike multiplication, where carry-save Wallace tree multipliers can be used for pipeline implementation.

Floating-Point Numbers

Representing Fractional Numbers

- A binary number with fractional part

$$B = b_{n-1} b_{n-2} \dots b_1 b_0 . b_{-1} b_{-2} \dots b_{-m}$$

corresponds to the decimal number

$$D = \sum_{i=-m}^{n-1} b_i 2^i$$

- Also called *fixed-point numbers*.
 - The position of the radix point is fixed.

If the radix point is allowed to move, we call it a floating-point representation.

Some Examples

$$1011.1 \quad \square \quad 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} = 11.5$$

$$101.11 \quad \square \quad 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2} = 5.75$$

$$10.111 \quad \square \quad 1 \times 2^1 + 0 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2} + 1 \times 2^{-3} = 2.875$$

Some Observations:

- Shift right by 1 bit means divide by 2
- Shift left by 1 bit means multiply by 2
- Numbers of the form $0.111111\dots_2$ has a value less than 1.0 (one).

Limitations of Representation

- In the fractional part, we can only represent numbers of the form $x/2^k$ exactly.
 - Other numbers have repeating bit representations (i.e. never converge).
- Examples:

$$3/4 = 0.11$$

$$7/8 = 0.111$$

$$5/8 = 0.101$$

$$1/3 = 0.10101010101 [01] \dots$$

$$1/5 = 0.001100110011 [0011] \dots$$

$$1/10 = 0.0001100110011 [0011] \dots$$

- More the number of bits, more accurate is the representation.
- We sometimes see: $(1/3)*3 \neq 1$.

Floating-Point Number Representation (IEEE-754)

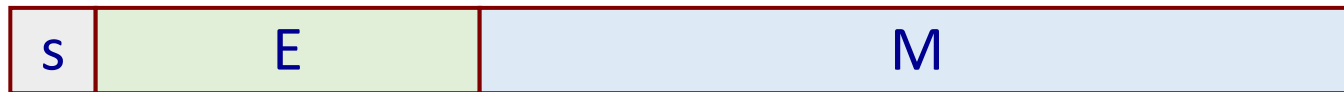
- For representing numbers with fractional parts, we can assume that the fractional point is somewhere in between the number (say, n bits in integer part, m bits in fraction part).
□ *Fixed-point representation*
 - Lacks flexibility.
 - Cannot be used to represent very small or very large numbers (for example: 2.53×10^{-26} , $1.7562 \times 10^{+35}$, etc.).
- **Solution** :: use *floating-point number representation*.
 - A number F is represented as a triplet $\langle s, M, E \rangle$ such that
$$F = (-1)^s M \times 2^E$$

$$F = (-1)^s M \times 2^E$$

- s is the *sign bit* indicating whether the number is negative ($=1$) or positive ($=0$).
- M is called the *mantissa*, and is normally a fraction in the range $[1.0, 2.0]$.
- E is called the *exponent*, which weights the number by power of 2.

Encoding:

- Single-precision numbers: total 32 bits, E 8 bits, M 23 bits
- Double-precision numbers: total 64 bits, E 11 bits, M 52 bits



Points to Note

- The number of *significant digits* depends on the number of bits in *M*.
 - 7 significant digits for 24-bit mantissa (23 bits + 1 implied bit).
- The *range* of the number depends on the number of bits in *E*.
 - 10^{38} to 10^{-38} for 8-bit exponent.

How many significant digits?

$$2^{24} = 10^x$$

$$24 \log_{10} 2 = x \log_{10} 10$$

$$x = 7.2 \quad \square \quad 7 \text{ significant decimal places}$$

Range of exponent?

$$2^{127} = 10^y$$

$$127 \log_{10} 2 = y \log_{10} 10$$

$$y = 38.1 \quad \square \quad \text{maximum exponent value} \\ 38 \text{ (in decimal)}$$

“Normalized” Representation

- We shall now see how E and M are actually encoded.
- Assume that the actual exponent of the number is EXP (i.e. number is $M \times 2^{EXP}$).
- Permissible range of E : $1 \leq E \leq 254$ (the all-0 and all-1 patterns are not allowed).
- Encoding of the exponent E :

The exponent is encoded as a biased value: $E = EXP + BIAS$

where $BIAS = 127$ ($2^{8-1} - 1$) for single-precision, and

$BIAS = 1023$ ($2^{11-1} - 1$) for double-precision.

- Encoding of the mantissa M:

- The mantissa is coded with an implied leading 1 (i.e. in 24 bits).

$$M = 1 . xxxx...x$$

- Here, xxxx...x denotes the bits that are actually stored for the mantissa. We get the extra leading bit for *free*.
- When xxxx...x = 0000...0, M is minimum (= 1.0).
- When xxxx...x = 1111...1, M is maximum (= 2.0 – ϵ).

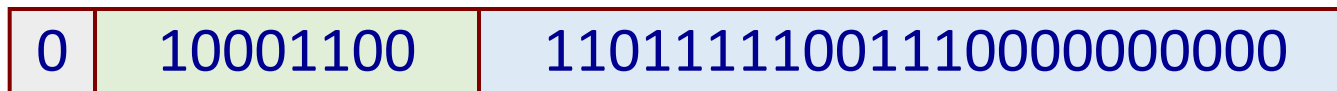
An Encoding Example

- Consider the number $F = 15335$

$$15335_{10} = 11101111100111_2 = 1.1101111100111 \times 2^{13}$$

- Mantissa will be stored as: $M = 1101111100111\ 0000000000_2$

- Here, $EXP = 13$, $BIAS = 127$. $\square$ $E = 13 + 127 = 140 = 10001100_2$



466F9C00 in hex

Another Encoding Example

- Consider the number $F = -3.75$

$$-3.75_{10} = -11.11_2 = -1.111 \times 2^1$$

- Mantissa will be stored as: $M = 111000000000000000000000_2$

- Here, $EXP = 1$, $BIAS = 127$. $\square$ $E = 1 + 127 = 128 = 10000000_2$



C0700000 in hex

Special Values

- When $E = 000\dots 0$

- $M = 000\dots 0$ represents the value 0.
- $M \neq 000\dots 0$ represents numbers very close to 0.

- When $E = 111\dots 1$

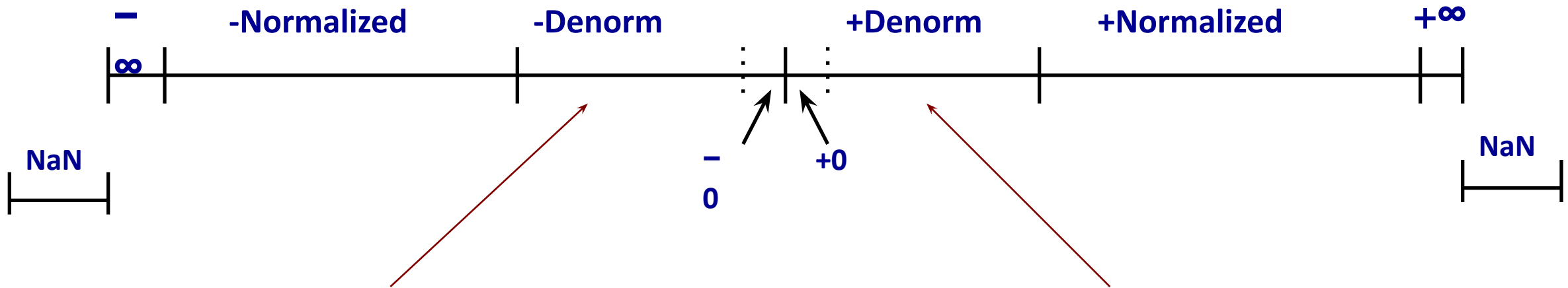
- $M = 000\dots 0$ represents the value ∞ (infinity).
- $M \neq 000\dots 0$ represents *Not-a-Number* (NaN).

Zero is represented by the *all-zero string*.

Also referred to as *de-normalized* numbers.

NaN represents cases when no numeric value can be determined, like uninitialized values, $\infty * 0$, $\infty - \infty$, square root of a negative number, etc.

Summary of Number Encodings



Denormal numbers have very small magnitudes (close to 0) such that trying to normalize them will lead to an exponent that is below the minimum possible value.

- Mantissa with leading 0's and exponent field equal to zero.
- Number of significant digits gets reduced in the process.

Rounding

- Suppose we are adding two numbers (say, in single-precision).
 - We add the mantissa values after shifting one of them right for exponent alignment.
 - We take the first 23 bits of the sum, and discard the residue R (remaining bits).
- IEEE-754 format supports four rounding modes:
 - a) Truncation
 - b) Round to $+\infty$ (similar to ceiling function)
 - c) Round to $-\infty$ (similar to floor function)
 - d) Round to nearest

- To implement rounding, two temporary bits are maintained:
 - *Round Bit (r)*: This is equal to the MSB of the residue R .
 - *Sticky Bit (s)*: This the logical OR of the rest of the bits of the residue R .
- Decisions regarding rounding can be taken based on these bits:
 - a) $R > 0$: if $r + s = 1$
 - b) $R = 0.5$: if $r.s' = 1$
 - c) $R > 0.5$: if $r.s = 1$ // '+' is logical OR, '.' is logical AND
- Renormalization after Rounding:
 - If the process of rounding generates a result that is not in normalized form, then we need to re-normalize the result.

Some Exercises

Decode the following single-precision floating-point numbers.

- a) 0011 1111 1000 0000 0000 0000 0000 0000
- b) 0100 0000 0110 0000 0000 0000 0000 0000
- c) 0100 1111 1101 0000 0000 0000 0000 0000
- d) 1000 0000 0000 0000 0000 0000 0000 0000
- e) 0111 1111 1000 0000 0000 0000 0000 0000
- f) 0111 1111 1101 0101 0101 0101 0101 0101

Floating-Point Arithmetic

Floating Point Addition/Subtraction

- Two numbers: $M1 \times 2^{E1}$ and $M2 \times 2^{E2}$, where $E1 > E2$ (say).
- Basic steps:
 - Select the number with the smaller exponent (i.e. $E2$) and shift its mantissa right by $(E1-E2)$ positions.
 - Set the exponent of the result equal to the larger exponent (i.e. $E1$).
 - Carry out $M1 \pm M2$, and determine the sign of the result.
 - Normalize the resulting value, if necessary.

Addition Example

- Suppose we want to add $F1 = 270.75$ and $F2 = 2.375$

$$F1 = (270.75)_{10} = (100001110.11)_2 = 1.0000111011 \times 2^8$$

$$F2 = (2.375)_{10} = (10.011)_2 = 1.0011 \times 2^1$$

- Shift the mantissa of F2 right by $8 - 1 = 7$ positions, and add:

$$\begin{array}{r}
 1000\ 0111\ 0110\ 0000\ 0000\ 0000 \\
 1\ 0011\ 0000\ 0000\ 0000\ 0000\ 000 \\
 \hline
 1000\ 1000\ 1001\ 0000\ 0000\ 0000\ 0000\ 000
 \end{array}$$

- **Result:** 1.00010001001×2^8

Residue

Subtraction Example

- Suppose we want to subtract $F2 = 224$ from $F1 = 270.75$

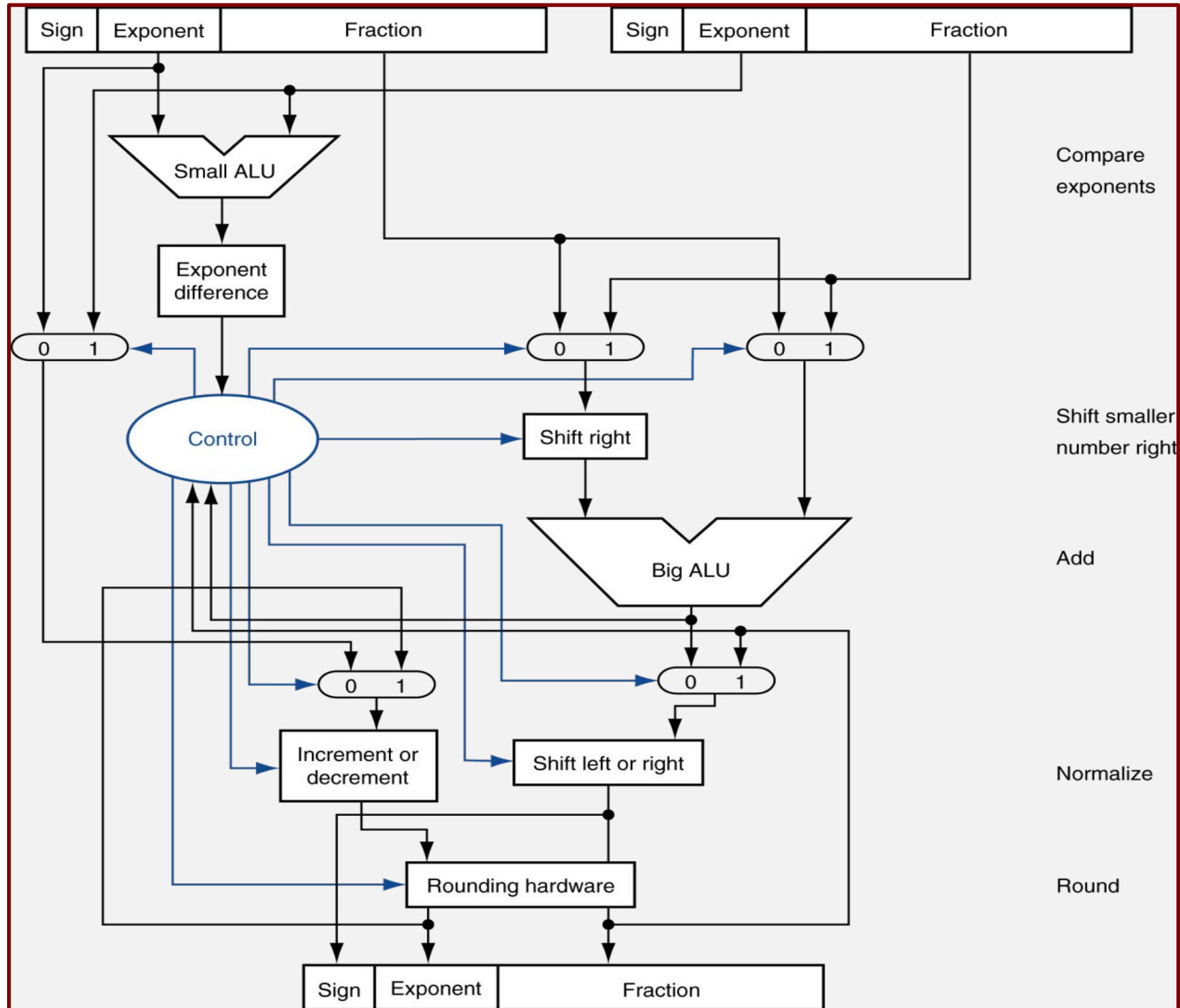
$$F1 = (270.75)_{10} = (100001110.11)_2 = 1.0000111011 \times 2^8$$

$$F2 = (224)_{10} = (11100000)_2 = 1.111 \times 2^7$$

- Shift the mantissa of $F2$ right by $8 - 7 = 1$ position, and subtract:

$$\begin{array}{r} 1000\ 0111\ 0110\ 0000\ 0000\ 0000 \\ 111\ 0000\ 0000\ 0000\ 0000\ 0000\ 000 \\ \hline 0001\ 0111\ 0110\ 0000\ 0000\ 0000\ 000 \end{array}$$

- For normalization, shift mantissa left 3 positions, and decrement E by 3.
- **Result:** 1.01110110×2^5



Floating-Point Multiplication

- Two numbers: $M1 \times 2^{E1}$ and $M2 \times 2^{E2}$
- Basic steps:
 - Add the exponents $E1$ and $E2$ and subtract the $BIAS$.
 - Multiply $M1$ and $M2$ and determine the *sign* of the result.
 - Normalize the resulting value, if necessary.

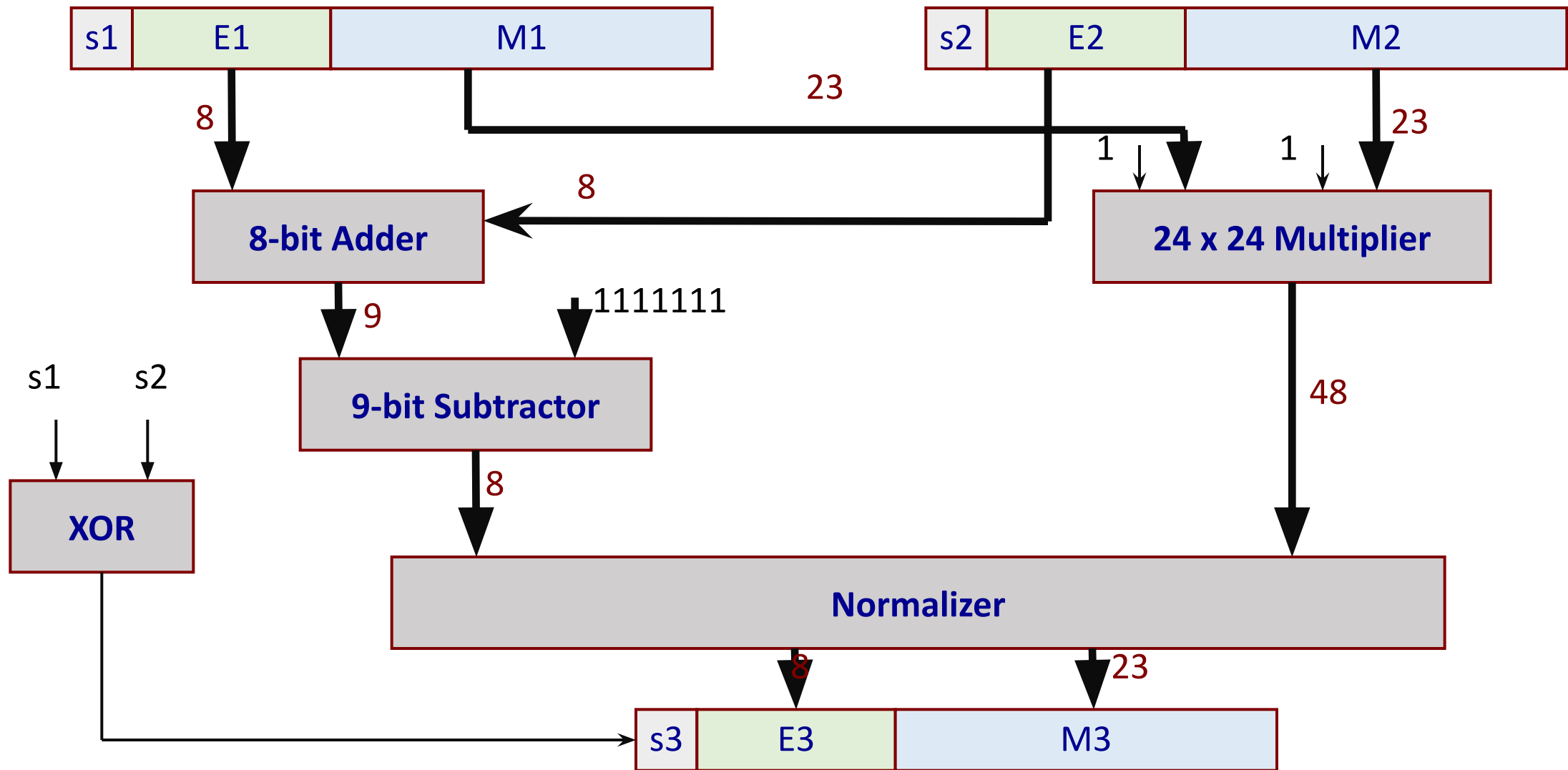
Multiplication Example

- Suppose we want to multiply $F1 = 270.75$ and $F2 = -2.375$

$$F1 = (270.75)_{10} = (100001110.11)_2 = 1.0000111011 \times 2^8$$

$$F2 = (-2.375)_{10} = (-10.011)_2 = -1.0011 \times 2^1$$

- Add the exponents: $8 + 1 = 9$
- Multiply the mantissas: 1.01000001100001
- **Result:** $- 1.01000001100001 \times 2^9$



Floating-Point Division

- Two numbers: $M1 \times 2^{E1}$ and $M2 \times 2^{E2}$
- Basic steps:
 - Subtract the exponents $E1$ and $E2$ and add the $BIAS$.
 - Divide $M1$ by $M2$ and determine the *sign* of the result.
 - Normalize the resulting value, if necessary.

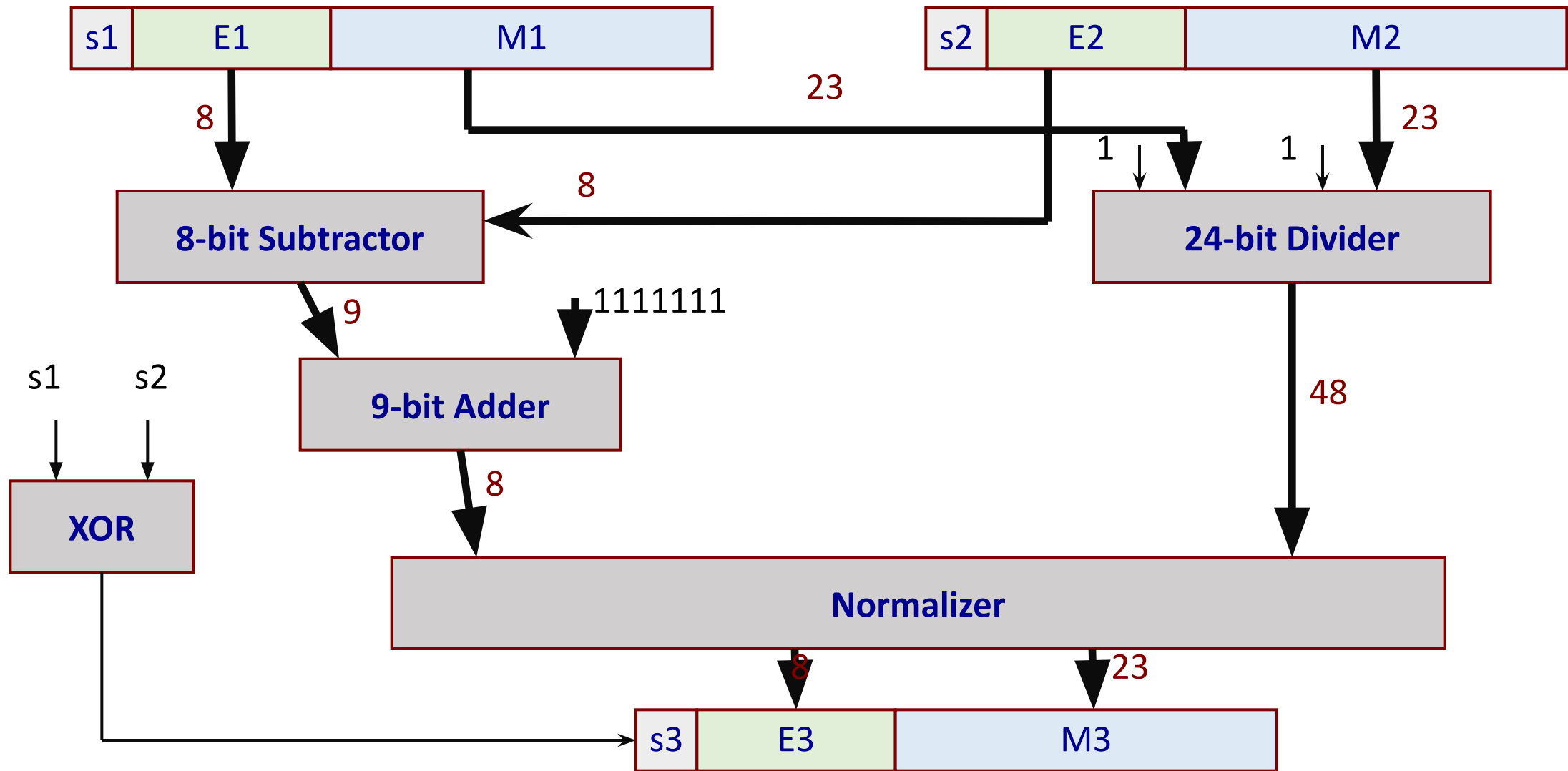
Division Example

- Suppose we want to divide $F1 = 270.75$ by $F2 = -2.375$

$$F1 = (270.75)_{10} = (100001110.11)_2 = 1.0000111011 \times 2^8$$

$$F2 = (-2.375)_{10} = (-10.011)_2 = -1.0011 \times 2^1$$

- Subtract the exponents: $8 - 1 = 7$
- Divide the mantissas: 0.1110010
- **Result:** $- 0.1110010 \times 2^7$
- **After normalization:** $- 1.110010 \times 2^6$



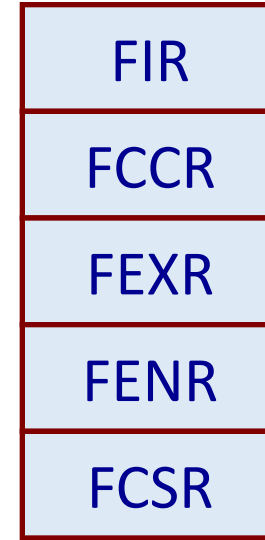
FLOATING-POINT ARITHMETIC in MIPS32

- The MIPS32 architecture defines the following floating-point registers (**FPRs**).
 - 32 32-bit floating-point registers **F0** to **F31**, each of which is capable of storing a single-precision floating-point number.
 - Double-precision floating-point numbers can be stored in even-odd pairs of FPRs (e.g., (**F0,F1**), (**F10,F11**), etc.).
- In addition, there are five *special-purpose FPU control registers*.

FPRs



**Special-purpose
Registers**



Typical Floating Point Instructions in MIPS32

- Load and Store instructions
 - Load Word from memory
 - Load Double-word from memory
 - Store Word to memory
 - Store Double-word to memory
- Data Movement instructions
 - Move data between integer registers and floating-point registers
 - Move data between integer registers and floating-point control registers

- Arithmetic instructions
 - Floating-point absolute value
 - Floating-point compare
 - Floating-point negate
 - Floating-point add
 - Floating-point subtract
 - Floating-point multiply
 - Floating-point divide
 - Floating-point square root
 - Floating-point multiply add
 - Floating-point multiply subtract

- Rounding instructions:
 - Floating-point truncate
 - Floating-point ceiling
 - Floating-point floor
 - Floating-point round
- Format conversions:
 - Single-precision to double-precision
 - Double-precision to single-precision

Example: Add a scalar s to a vector A

```
for (i=1000; i>0; i--)  
    A[i]= A[i] + s;
```

```
Loop:  L.D    F0,0(R1)  
        ADD.D  F4,F0,F2  
        S.D    F4,0(R1)  
        ADDI    R1,R1,-8  
        BNE    R1,R2,Loop
```

$R1$: initially points to $A[1000]$

$(F2,F3)$: contains the scalar s

$R2$: initialized such that $8(R2)$ is the
address of $A[1]$

We assume double precision (64 bits):

- Numbers stored in $(F0,F1)$, $(F2,F3)$, and $(F4,F5)$.